

+ 26

The Journey To /W4 /WX

How Hard Could It Be?

KEITH STOCKDALE



20
26



About me!



- Senior Software Engineer at Rare Ltd
- Started as an intern in 2015
- Work on the rendering and engine teams
- Participate in standardization

About me!



- Senior Software Engineer at Rare Ltd
- Started as an intern in 2015
- Work on the rendering and engine teams
- Participate in standardization... of HLSL (ECMA TC-57)



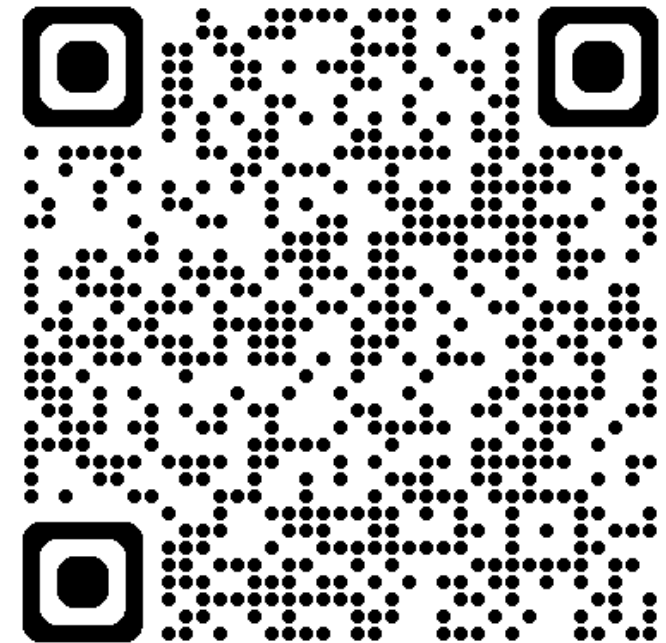
About Rare



About Rare



Upgrading to C++20



What did we achieve?



```
#include <iostream>
```

```
int main()
{
    std::cout << L"Hello CppCon!";
}
```

```
Execution build compiler returned: 0
Program returned: 0
00007FF66CC0D3E0
```

```
// The following deleted overloads prevent formatting strings as pointer values.
template<class traits>
basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, const wchar_t*) = delete;
```

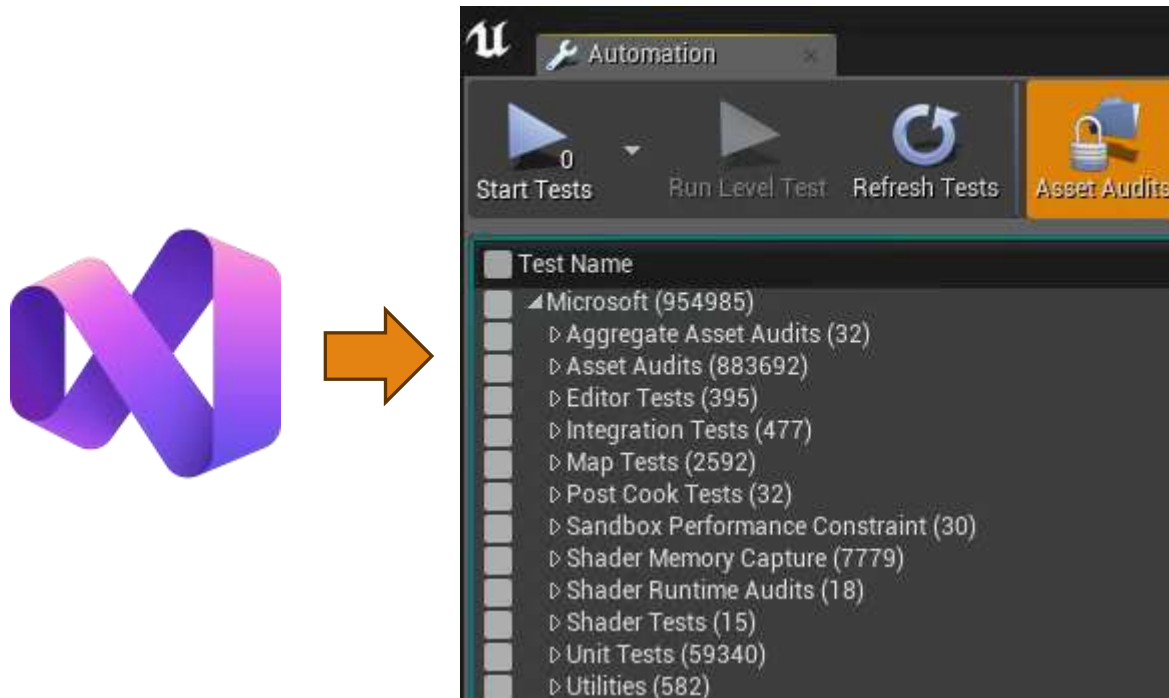
	/std:c++17 (mins:secs)	/std:c++17 /permissive- (mins:secs)	/std:c++20 (mins:secs)
Mean Rebuild Time	12:58.7	12:46.4	12:47.7

```
MyStruct MyObj
{
    .Name{ TEXT("Foo") },
    .Position{ 2.0, 3.14, 42.0 }
};
```

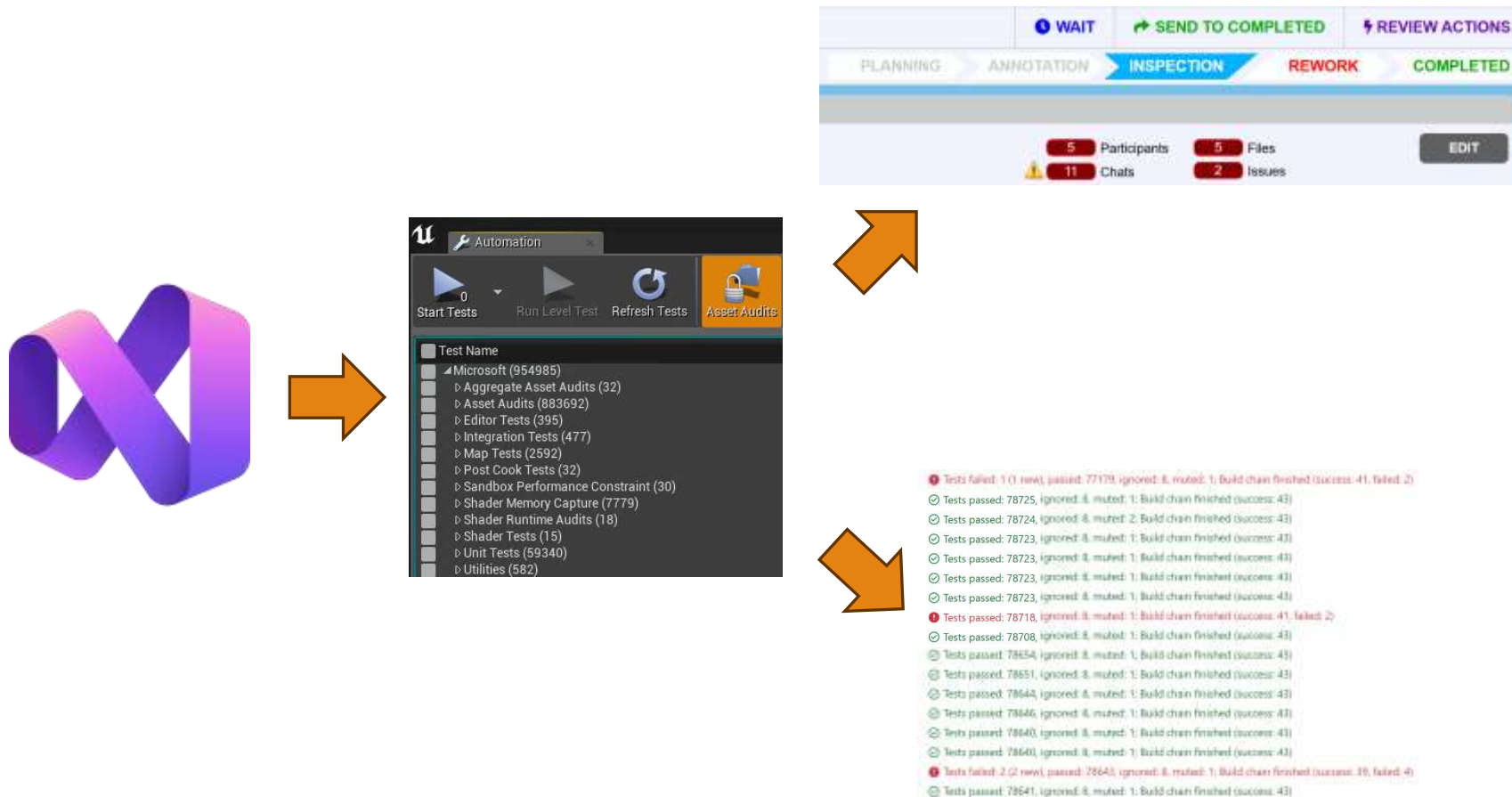
So why /W4 /WX?



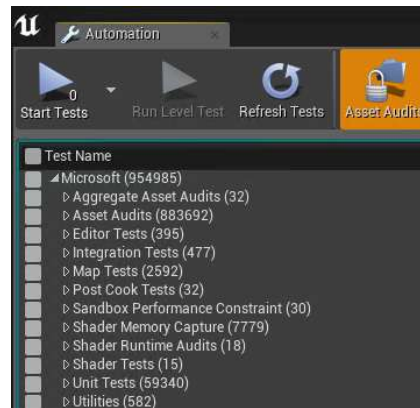
The journey of code at Rare



The journey of code at Rare

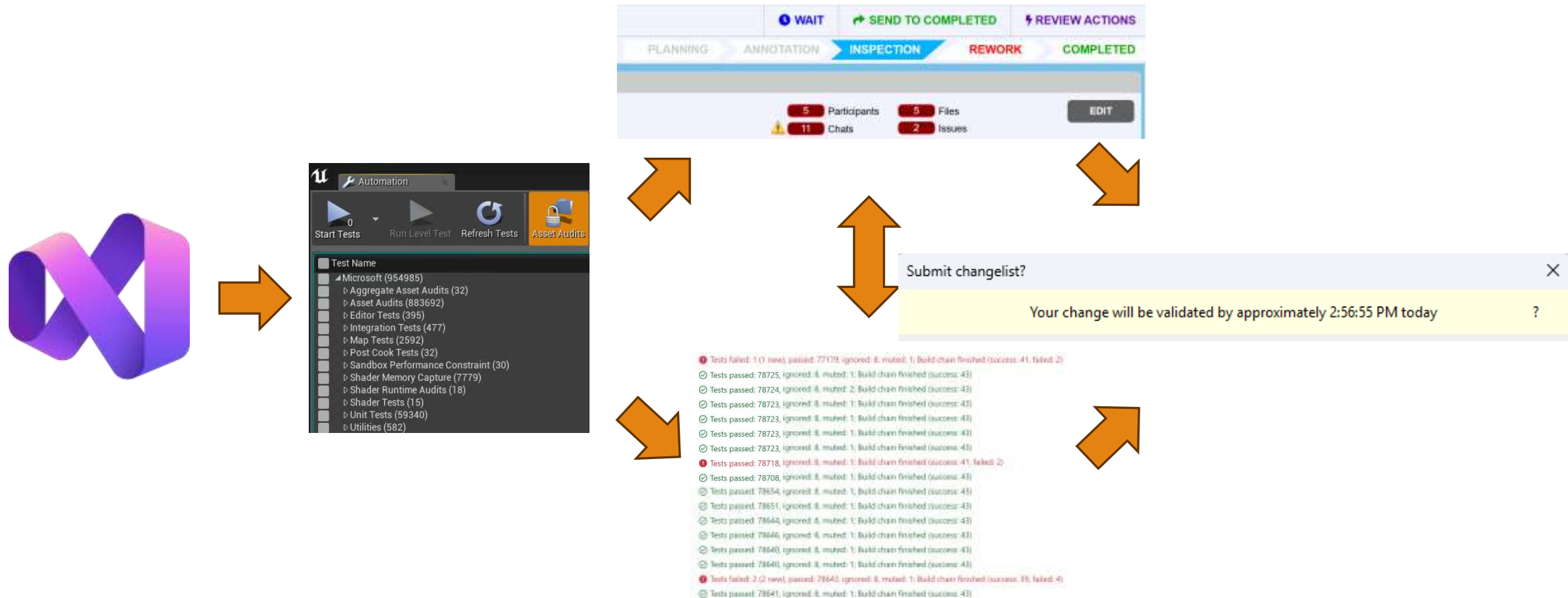


The journey of code at Rare

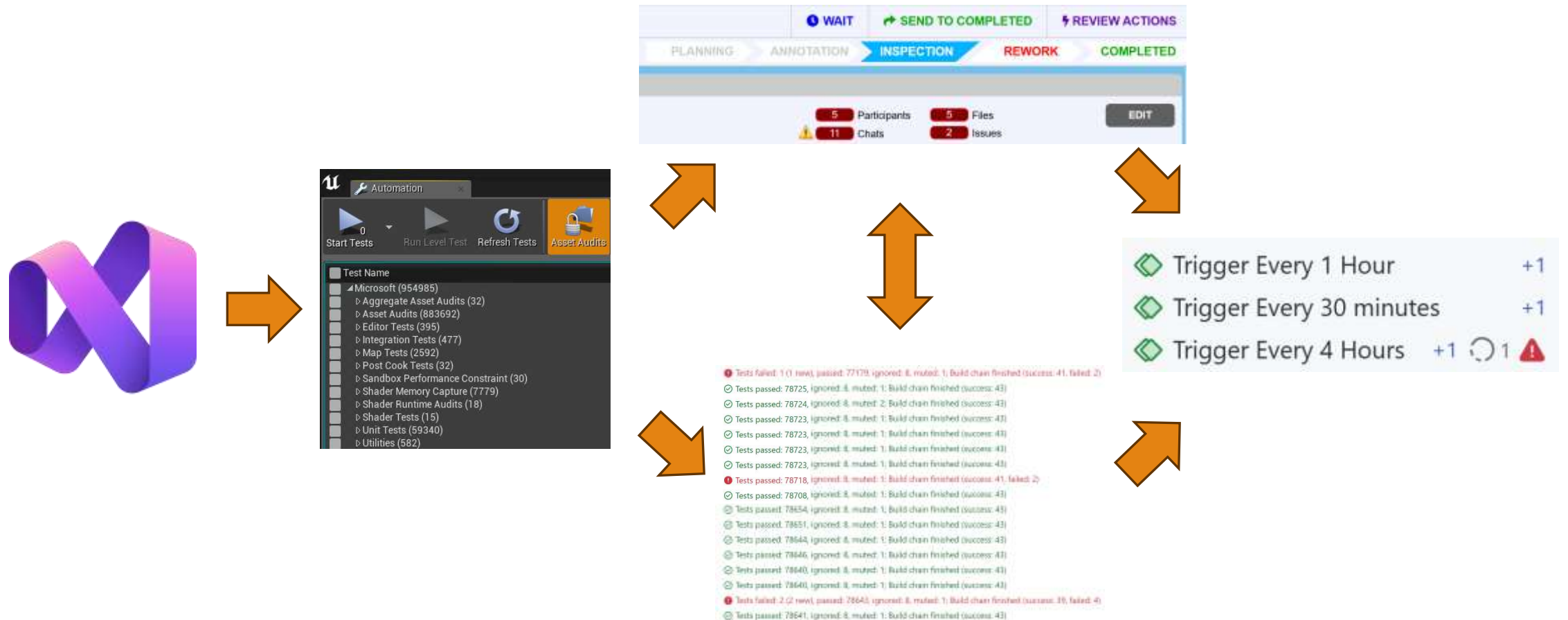


✓ Tests passed: 78725,
✓ Tests passed: 78724, is finished (success: 41, failed: 2)
✓ Tests passed: 78723, is: 43)
✓ Tests passed: 78723, is: 43)
✓ Tests passed: 78723, is: 43)
✓ Tests passed: 78723, is: 43)
✓ Tests passed: 78723, is: 43)
✓ Tests passed: 78723, is: 43)
! Tests passed: 78718, is: 43)
✓ Tests passed: 78708, is finished (success: 39, failed: 4)

The journey of code at Rare



The journey of code at Rare

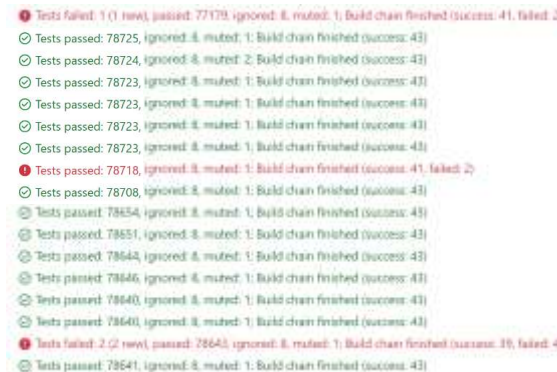
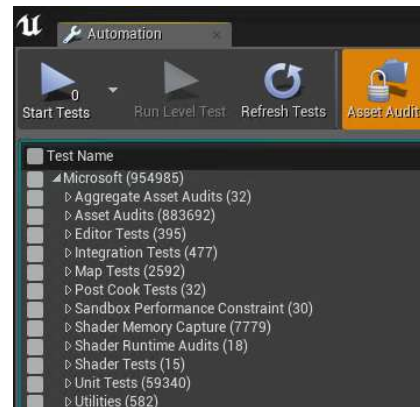


The journey of code at Rare

Cheap

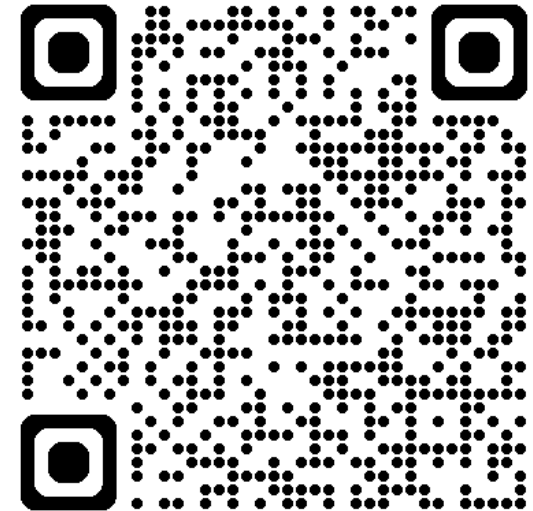
Issue Cost

Expensive



How do warnings help?

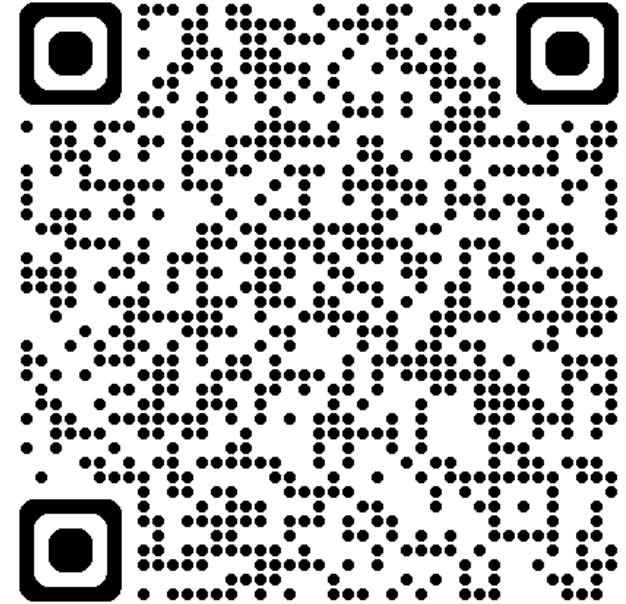
“Warnings are diagnostic messages that report constructions that are not inherently erroneous but that are risky or suggest there may have been an error.” – GCC docs



Where does /W4 come in?

“Use every available and reasonable set of warning options...”

“.../W4 = All reasonable warnings” –
Jason Turner



For the GCC/Clang users

/W0	Disable all warnings
/W1	Enable -Wall
/W2	Enable -Wall
/W3	Enable -Wall
/W4	Enable -Wall and -Wextra
/Wall	Enable -Weverything
/WX-	Do not treat warnings as errors
/WX	Treat warnings as errors

And why /WX?



The Engine team lead sends you this...

Stuart Milne (HE/HIM) 01/05 16:02



```
[2602/2918] OnlineSessionInterfaceLiveGDK.cpp
C:\Program Files (x86)\Microsoft GDK\2604400\GDK\Game\IT\Include\Xbox\invite.h(20): error C4768: __declspec attributes before linkage specification are ignored
C:\Program Files (x86)\Microsoft GDK\2604400\GDK\Game\IT\Include\Xbox\invite.h(28): error C4768: __declspec attributes before linkage specification are ignored
C:\Program Files (x86)\Microsoft GDK\2604400\GDK\Game\IT\Include\Xbox\invite.h(34): error C4768: __declspec attributes before linkage specification are ignored
C:\Program Files (x86)\Microsoft GDK\2604400\GDK\Game\IT\Include\Xbox\invite.h(42): error C4768: __declspec attributes before linkage specification are ignored
C:\Program Files (x86)\Microsoft GDK\2604400\GDK\Game\IT\Include\Xbox\invite.h(48): error C4768: __declspec attributes before linkage specification are ignored
C:\Program Files (x86)\Microsoft GDK\2604400\GDK\ExtensionLibraries\Xbox.Services.API.C\Include\cpprest/containerstream.h(452): warning C4996: 'stdext::checked_array_iter'
```

Getting this on GDK upgrade on Xbox. Before I go digging wondered if this lights a bulb for you

And was it **you** who turned C4768 on? 😊

Would we have cared if this warning was not promoted to an error?...

```
ionInterfaceLiveGDK.cpp
Microsoft GDK\260400\GRDK\GameKit\Include\XGameInvite.h(20): error C4768: __declspec attributes before linkage specification are ignored
Microsoft GDK\260400\GRDK\GameKit\Include\XGameInvite.h(28): error C4768: __declspec attributes before linkage specification are ignored
Microsoft GDK\260400\GRDK\GameKit\Include\XGameInvite.h(34): error C4768: __declspec attributes before linkage specification are ignored
Microsoft GDK\260400\GRDK\GameKit\Include\XGameInvite.h(42): error C4768: __declspec attributes before linkage specification are ignored
Microsoft GDK\260400\GRDK\GameKit\Include\XGameInvite.h(48): error C4768: __declspec attributes before linkage specification are ignored
Microsoft GDK\260400\GRDK\ExtensionLibraries\Xbox.Services.API.C\Include\cpprest/containerstream.h(452): warning C4996: 'stdext::checked_a
```

This warning has been in the GDK FOR
YEARS



Update 1 comes along...

April 2026 GDK Update 1

Notifications

XBOX Game Package Manager

Use the latest XBOX Game Package Manager (XGPM) UI release from GitHub when creating and uploading MSIXVC2 packages. This release includes multiple fixes. To download it, go to [releases - Microsoft/PackageUploader](#).

Issue fixes

System

Fixed a memory leak in the `XGameActivationAcceptPendingInvite` API. [61953160]

Fixed an issue where `XLaunchUri` failed with URIs longer than 21 characters. [62181431]

Fixed an issue in the `XGameInvite.h` and `XGameProtocol.h` GDK header files that could cause a C4768 compiler error. [62181564]



Update 1 comes along...

April 2026 GDK Update 1

Notifications

XBOX Game Package Manager

Use the latest XBOX Game Package Manager (XGPM) UI release from GitHub when creating and uploading MSXVC2 packages. This release includes multiple fixes. To download it, see [releases](#) - [Microsoft Package Installer](#).

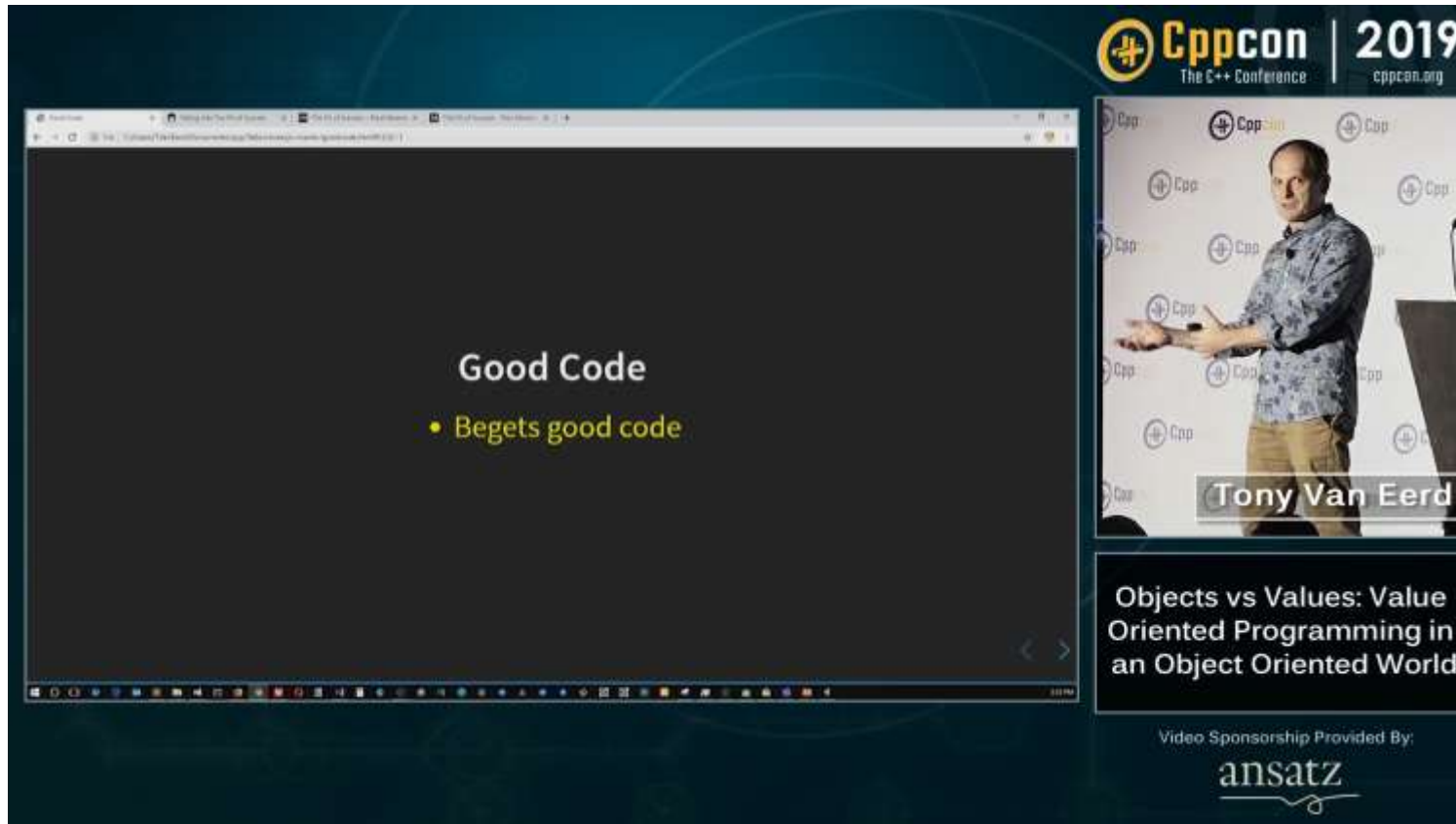
Fixed an issue in the `XGameInvite.h` and `XGameProtocol.h` GDK header files that could cause a C4768 compiler error. [62181564]

Fixed a memory leak in the `XGameInviteLinkAcceptingInvite` API. [51953180]

Fixed an issue where `XLaunchUI` failed with URIs longer than 21 characters. [62181431]

Fixed an issue in the `XGameInvite.h` and `XGameProtocol.h` GDK header files that could cause a C4768 compiler error. [62181564]

“Good code begets good code”



The image is a screenshot of a video player interface. The main content area displays a presentation slide with a dark background. The slide has the title "Good Code" in white text, followed by a bullet point "• Begets good code" in yellow text. To the right of the slide, there is a video feed of a speaker, Tony Van Eerd, standing at a podium. Above the video feed, the text "Cppcon | 2019" and "The C++ Conference" are visible, along with the website "cppcon.org". Below the video feed, the text "Objects vs Values: Value Oriented Programming in an Object Oriented World" is displayed. At the bottom of the video player, it says "Video Sponsorship Provided By: ansatz".

Good Code

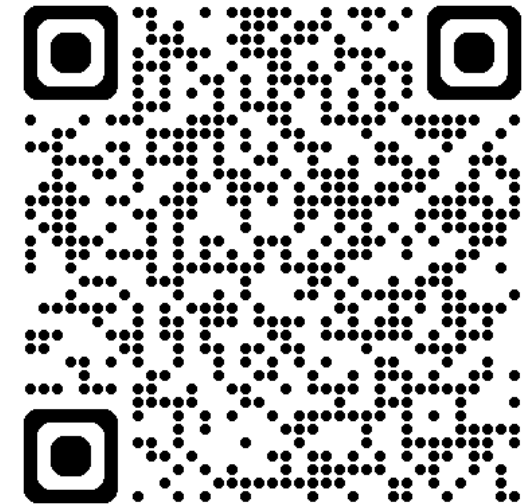
- Begets good code

Cppcon | 2019
The C++ Conference
cppcon.org

Tony Van Eerd

Objects vs Values: Value Oriented Programming in an Object Oriented World

Video Sponsorship Provided By:
ansatz



/W4 /WX shifts the load on to the compiler

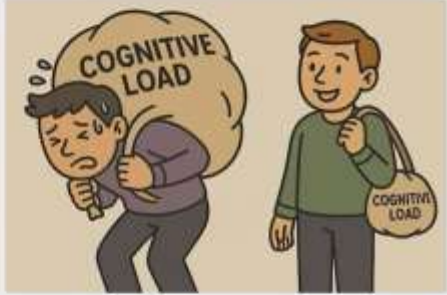
**Cppcon**
The C++ Conference
Cppcon.org



Code Review Preparation

Take it easy on your reviewer(s)

- Do a self review first
 - Reviewers should not be catching obvious / careless errors
 - Fix things that don't look right
- Reduce cognitive load
 - Limit the size/scope
 - Don't sneak in unrelated fixes
 - Expressive code and comments
 - Highlight where key changes are located

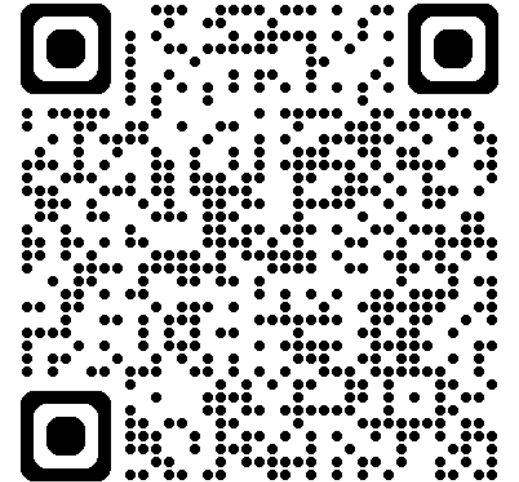


Bloomberg Image generated by ChatGPT/DAVINCI 26

Mastering the Code Review Process

Peter Muldoon





High warning levels are good practice...

Correct by Construction:
APIs That Are
Easy to Use and
Hard to Misuse

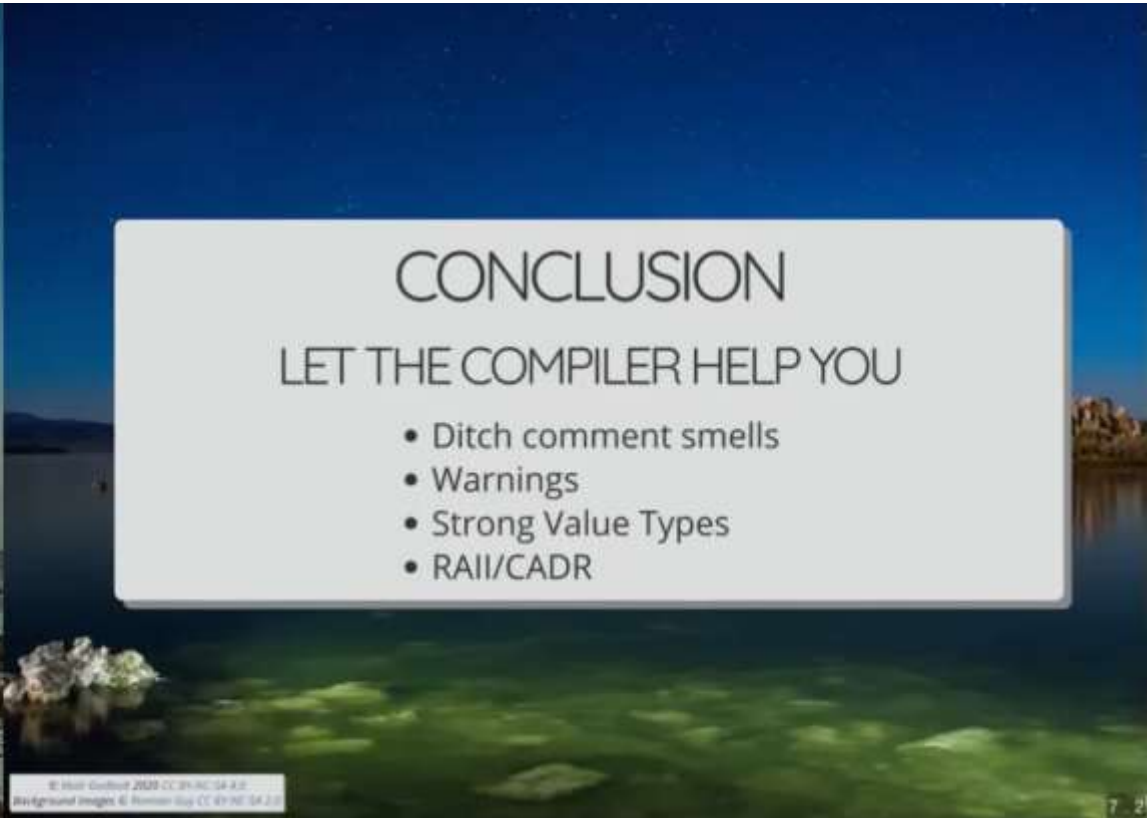


Matt Godbolt

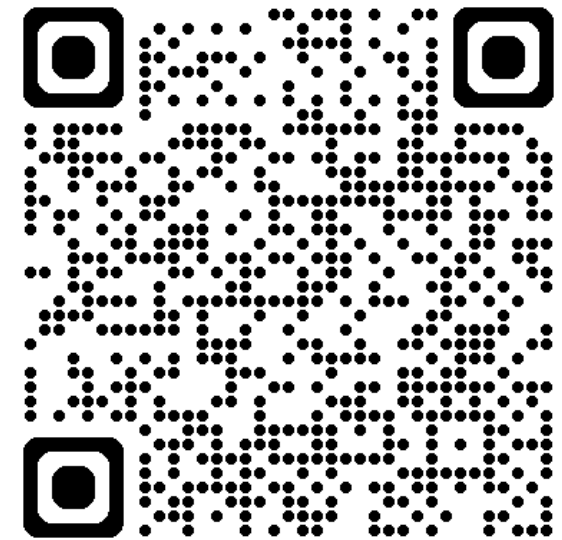
CONCLUSION

LET THE COMPILER HELP YOU

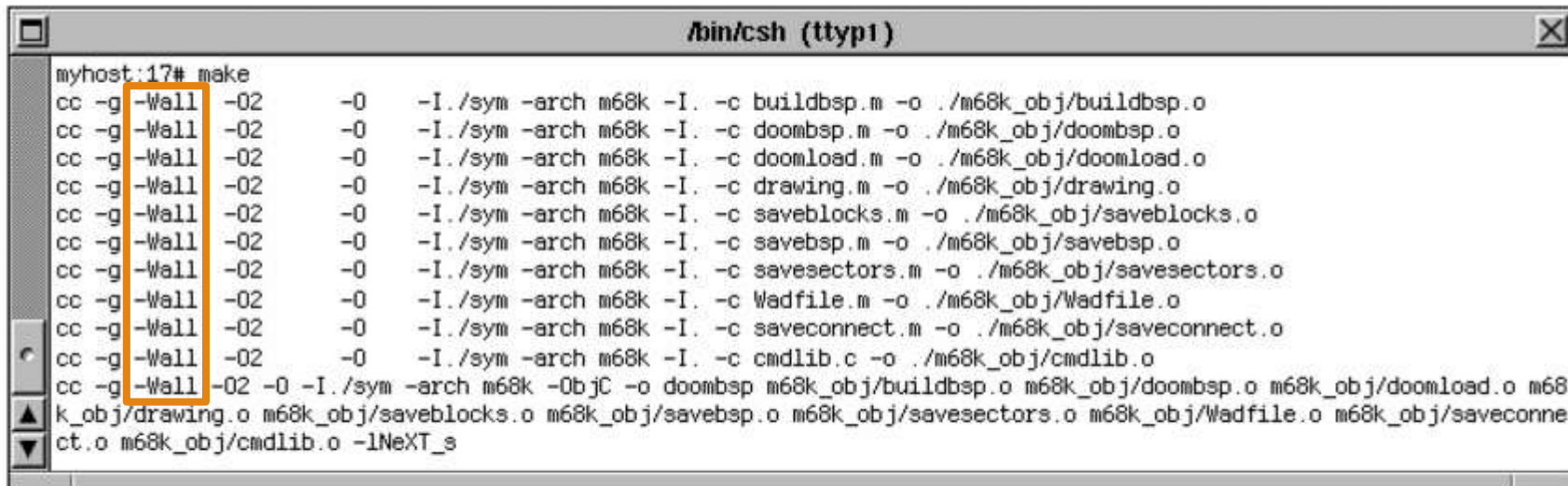
- Ditch comment smells
- Warnings
- Strong Value Types
- RAII/CADR



© Matt Godbolt 2020 CC BY-NC-SA 4.0
Background images © Remco de Vries CC BY-NC-SA 4.0



... and always have been...



```
myhost:17# make
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c buildbsp.m -o ./m68k_obj/buildbsp.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c doombsp.m -o ./m68k_obj/doombsp.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c doomload.m -o ./m68k_obj/doomload.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c drawing.m -o ./m68k_obj/drawing.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c saveblocks.m -o ./m68k_obj/saveblocks.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c savebsp.m -o ./m68k_obj/savebsp.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c savesectors.m -o ./m68k_obj/savesectors.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c Wadfile.m -o ./m68k_obj/Wadfile.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c saveconnect.m -o ./m68k_obj/saveconnect.o
cc -g -Wall -O2 -O -I./sym -arch m68k -I. -c cmdlib.c -o ./m68k_obj/cmdlib.o
cc -g -Wall -O2 -O -I./sym -arch m68k -ObjC -o doombsp m68k_obj/buildbsp.o m68k_obj/doombsp.o m68k_obj/doomload.o m68k_obj/drawing.o m68k_obj/saveblocks.o m68k_obj/savebsp.o m68k_obj/savesectors.o m68k_obj/Wadfile.o m68k_obj/saveconnect.o m68k_obj/cmdlib.o -lNeXT_s
```

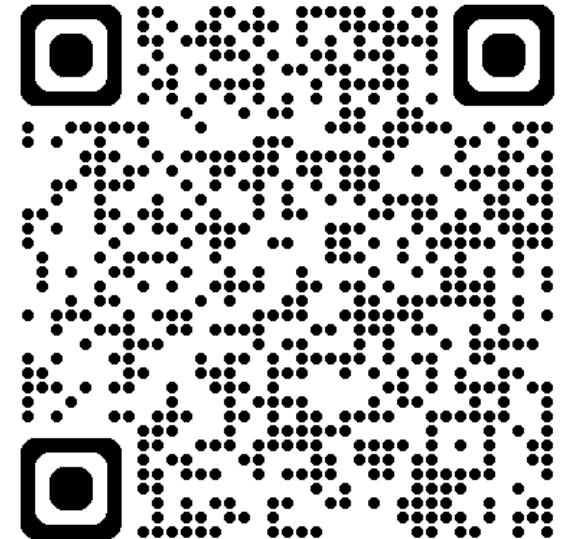
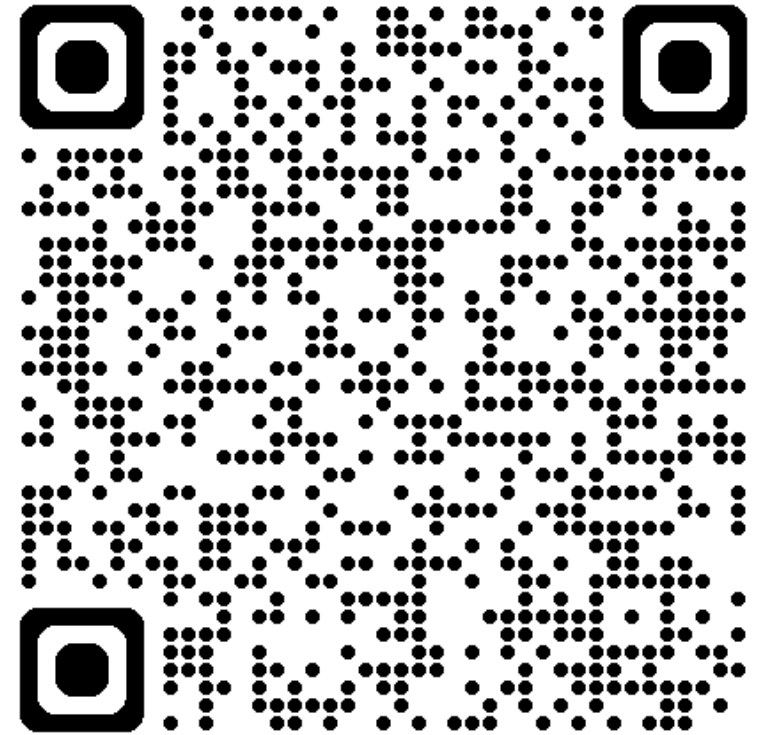


Figure 4.11: Building doombsp on NeXTSTEP.

... but can be tricky

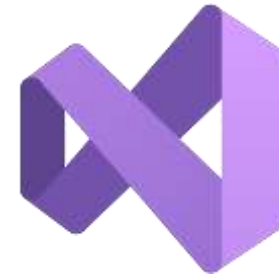
“Start with very strict warning settings from the beginning. **Trying to raise the warning level after the project is underway can be painful**” – Jason Turner



Key information about our code base

November 11, 2015

Unreal Engine 4.10



Sea of Thieves' code base

	Sea of Thieves project	Engine
Number of C++ files	29,936	19,580
Lines of Code	3,251,088	3,475,065

Aside: Latest UE's code base

	Sea of Thieves project	Sea of Thieves Engine	Latest Unreal Engine (ue5-main)
Number of C++ files	29,936	19,580	85,776
Lines of Code	3,251,088	3,475,065	15,638,565

Code base growth in a year

	Sea of Thieves project	Engine
Number of C++ files	1,667	161
Lines of Code	273,940	26,037

Plan of action

Step 0: What was our warning level?

```
// Set warning level.
if (Target.WindowsPlatform.Compiler != WindowsCompiler.Intel)
{
    // Restrictive during regular compilation.
    Arguments.Add("/W4");
}
else
{
    // If we had /W4 with clang or Intel on windows we would be flooded
    with warnings. This will be fixed incrementally.
    Arguments.Add("/W0");
}
```

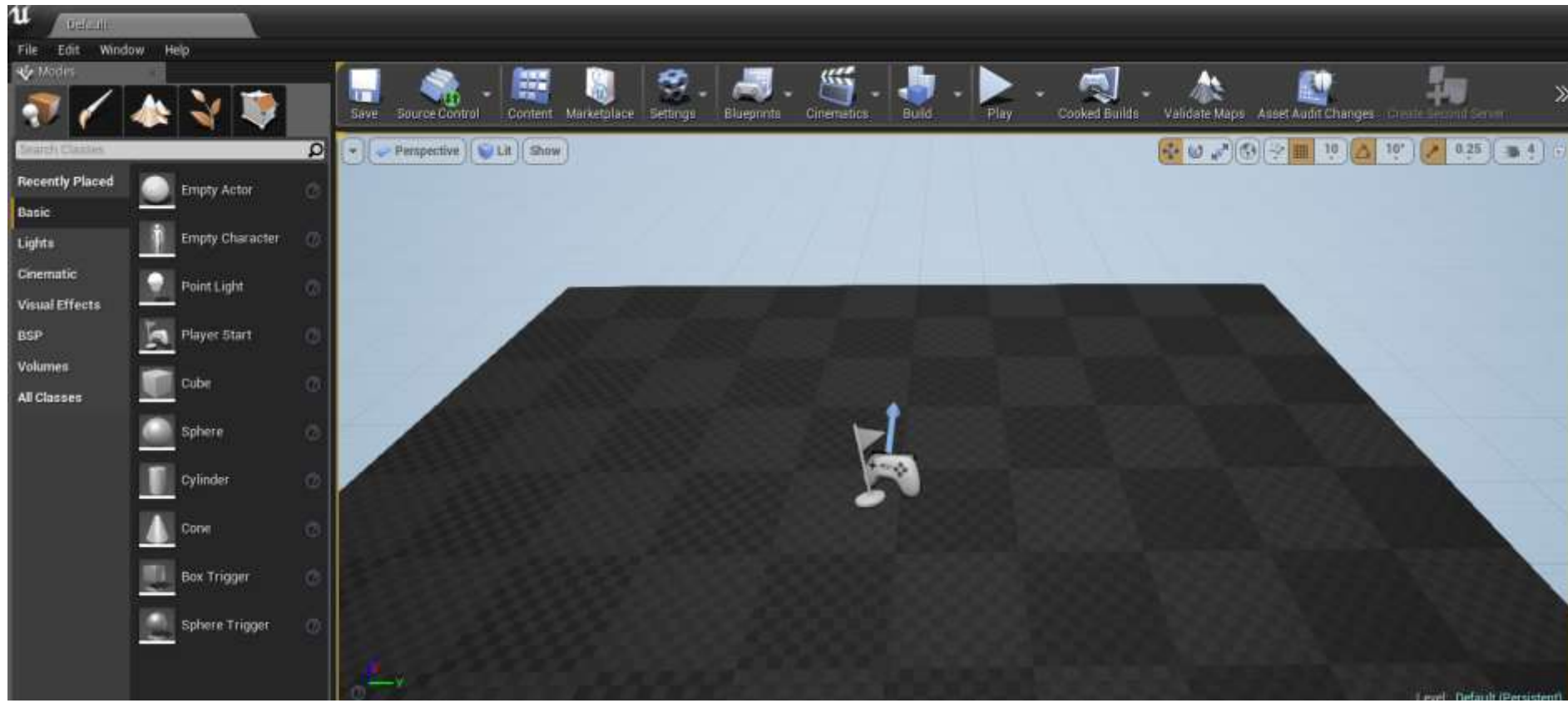
We are already at /W4!



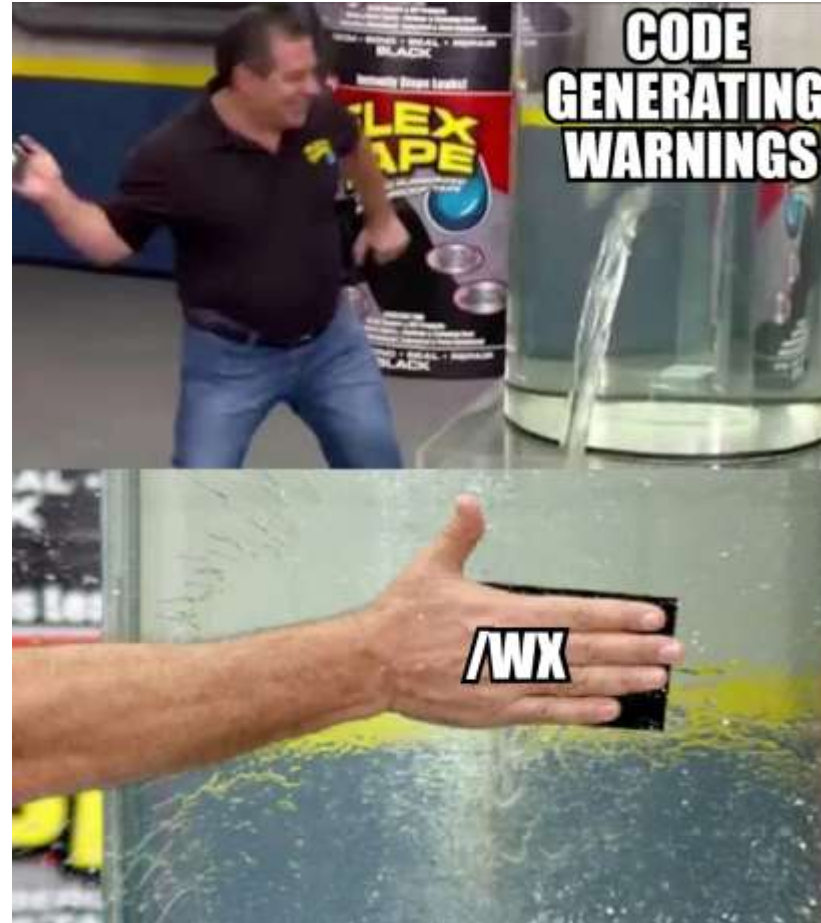
Step 1: Fix all the warnings in the editor



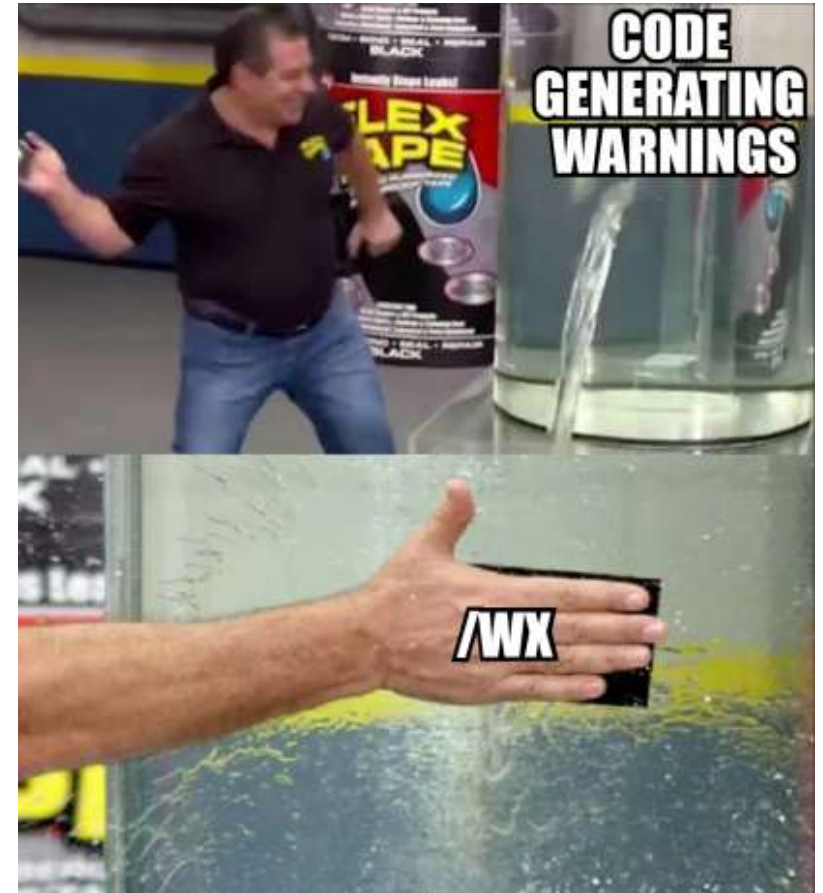
Unreal Engine Editor



Step 2: Enable /WX for the editor



Step 3: Repeat for Server and Client





Code Review #1

```
class FDelegateHandle
{
public:

    explicit FDelegateHandle(EGenerateNewHandleType)
        : ID(GenerateNewID())
    {
    }

    uint32 GetID() const
    {
        return ID;
    }

private:

};
```

Code Review #1

```
class FDelegateHandle
{
public:

    explicit FDelegateHandle(EGenerateNewHandleType)
        : ID(GenerateNewID())
    {
    }

    uint32 GetID() const
    {
        return ID;
    }

private:

    uint64 ID;
};
```

C4244 / -Wconversion

Compiler Warning (levels 3 and 4) C4244

✦ Summarize this article for me

'conversion' conversion from 'type1' to 'type2', possible loss of data

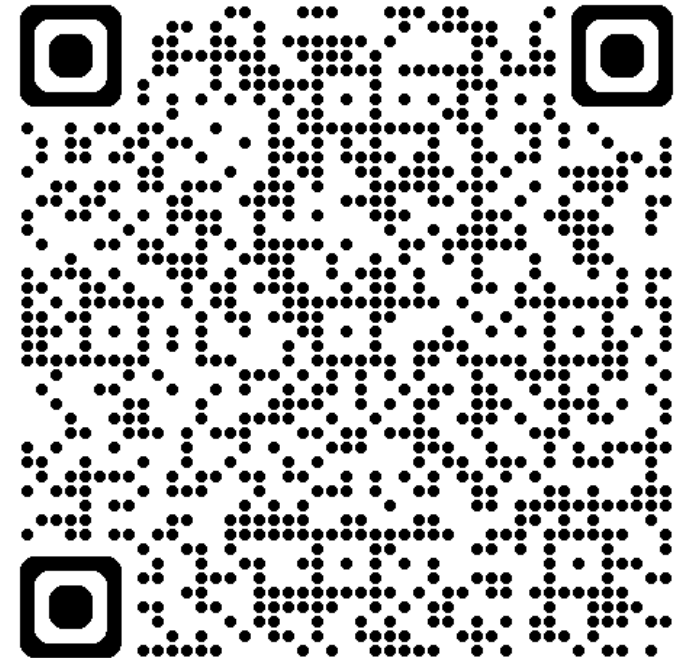
Remarks

An integer type is converted to a smaller integer type.

- This is a level-4 warning if *type1* is a signed or unsigned `int` and *type2* is a smaller, such as a signed or unsigned `short`.
- It's a level 3 warning if a value of type `__int64` or `unsigned __int64` is assigned to a signed or unsigned `int`. A possible loss of data may have occurred due to a narrowing conversion, which might lead to unexpected results.

To fix this warning, either change your program to use compatible types, or add logic that ensures that the range of possible values is compatible with the types you're using. If the conversion is intended, use an explicit cast to silence the warning.

C4244 can also appear when the warning level is 2. For more information, see [Compiler Warning \(level 2\) C4244](#).



Code Review #1

```
class FDelegateHandle
{
public:

    explicit FDelegateHandle(EGenerateNewHandleType)
        : ID(GenerateNewID())
    {
    }

    uint32 GetID() const
    {
        return ID;
    }

private:

    uint64 ID;
};
```

Code Review #1 – Potential “fix”

```
class FDelegateHandle
{
public:

    explicit FDelegateHandle(EGenerateNewHandleType)
        : ID(GenerateNewID())
    {
    }

    uint32 GetID() const
    {
        return static_cast<uint32>(ID);
    }

private:

    uint64 ID;
};
```

Code Review #1 – Potential fix

```
class FDelegateHandle
{
public:

    explicit FDelegateHandle(EGenerateNewHandleType)
        : ID(GenerateNewID())
    {
    }

    uint32 GetID() const
    {
        return ID;
    }

private:

    uint32 ID;
};
```

Code Review #1 – Actual fix

```
class FDelegateHandle
{
public:

    explicit FDelegateHandle(EGenerateNewHandleType)
        : ID(GenerateNewID())
    {
    }

    uint64 GetID() const
    {
        return ID;
    }

private:

    uint64 ID;
};
```

Code Review #2

```
enum MyEnum;
```

```
struct A  
{  
    MyEnum mem;  
};
```

```
enum MyEnum  
{  
    One,  
    Two  
};
```

```
int main()  
{  
    return sizeof(A);  
}
```

Code Review #2

```
enum MyEnum;
```

```
struct A  
{  
    MyEnum mem;  
};
```

```
enum MyEnum : char  
{  
    One,  
    Two  
};
```

```
int main()  
{  
    return sizeof(A);  
}
```

Code Review #2

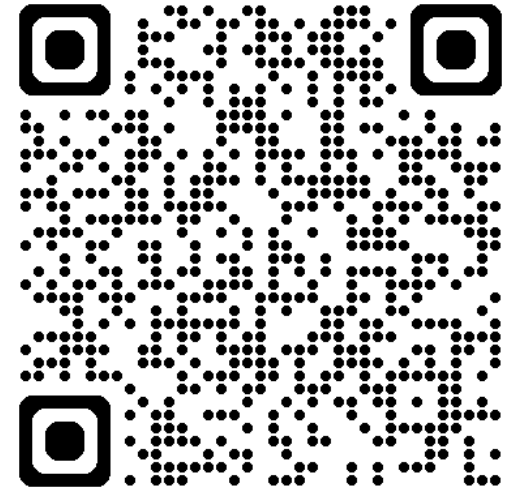
```
enum MyEnum;  
  
struct A  
{  
    MyEnum mem;  
};  
  
enum MyEnum : char  
{  
    One,  
    Two  
};
```

```
int main()  
{  
    if (sizeof(A) == sizeof(MyEnum))  
    {  
        return 1;  
    }  
    return 0;  
}
```

Output of x64 msvc v19.51 VS18.6 (Compiler #2) ✎ ✕

A ▾ ☐ Wrap lines ☐ Select all

example.cpp
ASM generation compiler returned: 0
example.cpp
Execution build compiler returned: 0
Program returned: 0



C4471

Compiler Warning (level 4) C4471



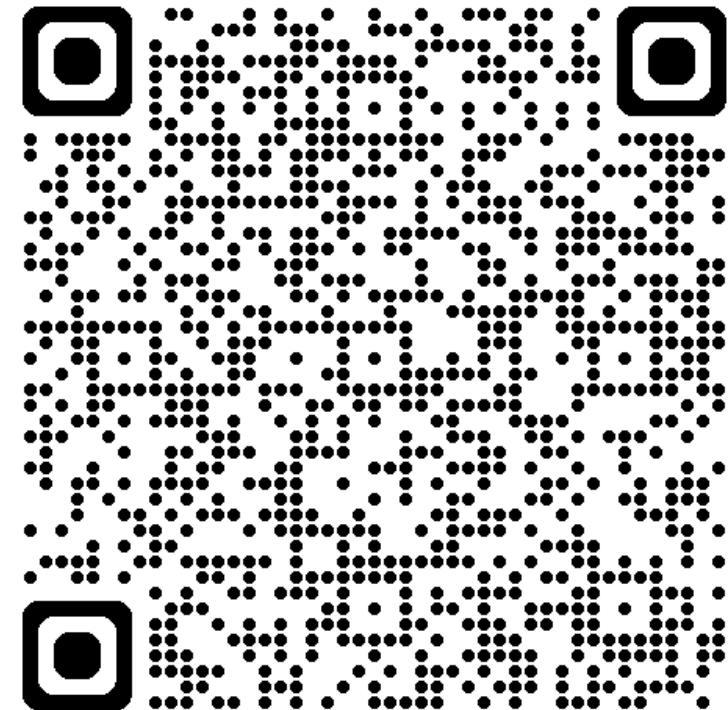
Summarize this article for me

'*enumeration*': a forward declaration of an unscoped enumeration must have an underlying type (int assumed)

Remarks

A forward declaration of an unscoped enumeration was found without a specifier for the underlying type. By default, Visual C++ assumes `int` is the underlying type for an enumeration. This can cause issues if a different type is used in the enumeration definition, for example, if a different explicit type is specified, or if a different type is implicitly set by an initializer. You may also have portability issues; other compilers do not assume `int` is the underlying type of an enumeration.

This warning is off by default; you can use `/Wall` or `/wN4471` to enable it on the command line, or use `#pragma warning` in your source file.



Code Review #2

```
enum MyEnum;
```

```
struct A  
{  
    MyEnum mem;  
};
```

```
enum MyEnum : char  
{  
    One,  
    Two  
};
```

```
int main()  
{  
    return sizeof(A);  
}
```

Code Review #2 – Potential Fix

```
enum MyEnum : char;
```

```
struct A  
{  
    MyEnum mem;  
};
```

```
enum MyEnum : char
```

```
{  
    One,  
    Two  
};
```

```
int main()  
{  
    return sizeof(A);  
}
```

Code Review #2 – Actual Fix

```
#include "MyEnum.h"
```

```
struct A  
{  
    MyEnum mem;  
};
```

```
int main()  
{  
    return sizeof(A);  
}
```

Code Review #3

```
using TargetFuncSig = int(float, float);
using TargetFuncSigPtr = TargetFuncSig*;

int Foo(HMODULE InModule)
{
    auto func = reinterpret_cast< TargetFuncSigPtr >(
        GetProcAddress(InModule, "MyFunc")
    );
    return func(3.14f, 2.0f);
}
```

GetProcAddress

```
WINBASEAPI  
FARPROC  
WINAPI  
GetProcAddress(  
    _In_ HMODULE hModule,  
    _In_ LPCSTR lpProcName  
);
```

GetProcAddress

```
typedef INT_PTR (FAR WINAPI *FARPROC)();  
  
WINBASEAPI  
FARPROC  
WINAPI  
GetProcAddress(  
    _In_ HMODULE hModule,  
    _In_ LPCSTR lpProcName  
);
```

GetProcAddress

```
typedef __int64          INT_PTR;  
  
typedef INT_PTR (FAR WINAPI *FARPROC)();  
  
WINBASEAPI  
FARPROC  
WINAPI  
GetProcAddress(  
    _In_ HMODULE hModule,  
    _In_ LPCSTR lpProcName  
);
```

C4191/ -Wcast-function-type-strict

Compiler Warning (level 3, off) C4191

✦ Summarize this article for me

'operation': unsafe conversion from 'type_of_expression' to 'type_required'
Making a function call using the resulting pointer may cause your program to fail

Remarks

Several operations involving function pointers are considered unsafe:

- Function types with different calling conventions.
- Function types with different return conventions.
- Argument or return types with different sizes, type categories, or classifications.
- Different argument list lengths (on `__cdecl`, only on cast from longer list to shorter list, even if shorter is varargs).
- Pointer to data (other than `void*`) aliased against a pointer to function.
- Any other type difference that would yield an error or warning on a `reinterpret_cast`.

Calling this function through the result pointer might cause your program to crash.

This warning is off by default. For more information, see [Compiler Warnings That Are Off by Default](#).



If we could have nice things...

```
WINBASEAPI  
FARPROC  
WINAPI  
GetProcAddress(  
    _In_ HMODULE hModule,  
    _In_ LPCSTR lpProcName  
);
```

If we could have nice things...

```
WINBASEAPI  
void*  
WINAPI  
GetProcAddress(  
    _In_ HMODULE hModule,  
    _In_ LPCSTR lpProcName  
);
```

Code Review #3 - Actual Fix

```
using TargetFuncSig = int(float, float);
using TargetFuncSigPtr = TargetFuncSig*;

int Foo(HMODULE InModule)
{
    auto func = reinterpret_cast< TargetFuncSigPtr >(
        GetProcAddress(InModule, "MyFunc")
    );
    return func(3.14f, 2.0f);
}
```

Code Review #3 - Actual Fix

```
using TargetFuncSig = int(float, float);
using TargetFuncSigPtr = TargetFuncSig*;

int Foo(HMODULE InModule)
{
#pragma warning( push )
#pragma warning(disable : 4191)
    auto func = reinterpret_cast< TargetFuncSigPtr >(
        GetProcAddress(InModule, "MyFunc")
    );
#pragma warning(pop)
    return func(3.14f, 2.0f);
}
```

Code Review #3 - Best Fix

```
using TargetFuncSig = int(float, float);
using TargetFuncSigPtr = TargetFuncSig*;

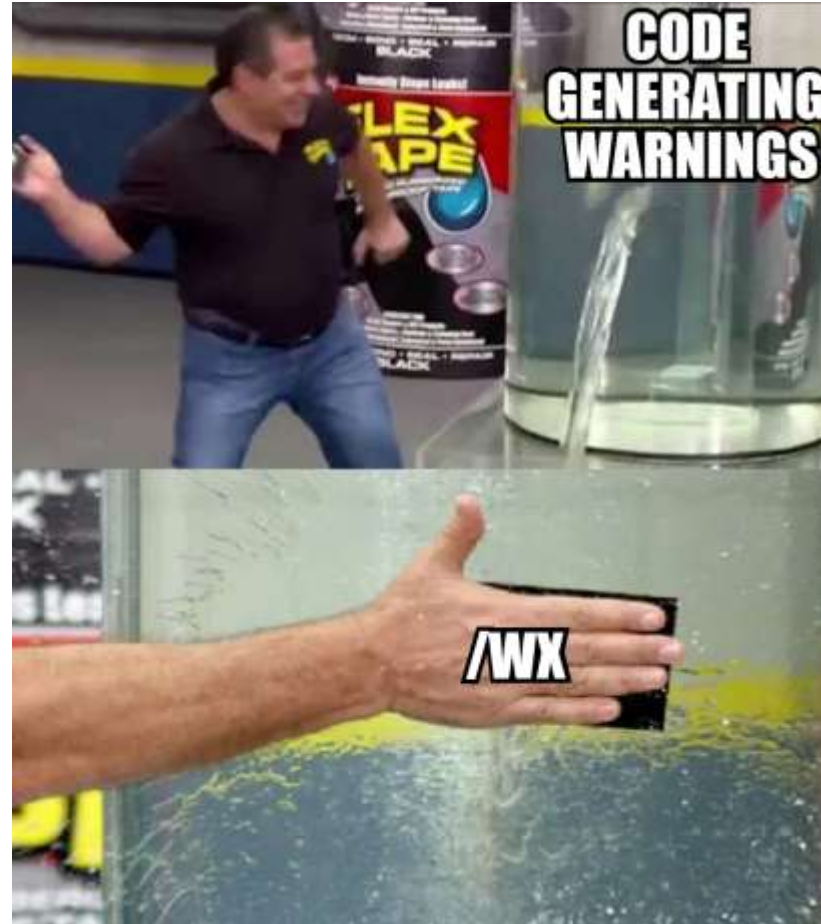
int Foo(HMODULE InModule)
{
    // GetProcAddress returns a ptr to int64(). So, we need
    // to reinterpret cast.. and silence the warning telling
    // us that this is a bad thing to do.. Sorry!
    #pragma warning( push )
    #pragma warning(disable : 4191)
        auto func = reinterpret_cast< TargetFuncSigPtr >(
            GetProcAddress(InModule, "MyFunc")
        );
    #pragma warning(pop)
    return func(3.14f, 2.0f);
}
```

... Are we done?

Build succeeded.
0 Warning(s)
0 Error(s)



Step 2?



On to the server then?



Code Review #4

```
void OnlineGameMode::EndSession()
{
    UnregisterEvents();

    if (GetRole() != NM_DedicatedServer)
    {
        return;
    }

    TelemetryInterface.EndSession();
}
```

Code Review #4

```
enum ENetMode
```

```
{
```

```
    NM_StandAlone,  
    NM_DedicatedServer,  
    NM_ListenServer,  
    NM_Client,  
    NM_MAX,
```

```
};
```

```
enum ENetRole
```

```
{
```

```
    ROLE_None,  
    ROLE_SimulatedProxy,  
    ROLE_AutonomousProxy,  
    ROLE_Authority,  
    ROLE_MAX,
```

```
};
```

```
void OnlineGameMode::EndSession()
```

```
{
```

```
    UnregisterEvents();
```

```
    if (GetRole() != NM_DedicatedServer)
```

```
    {
```

```
        return;
```

```
    }
```

```
    TelemetryInterface.EndSession();
```

```
}
```

Code Review #4

```
enum ENetMode
```

```
{
```

```
    NM_StandAlone,  
    NM_DedicatedServer,  
    NM_ListenServer,  
    NM_Client,  
    NM_MAX,
```

```
};
```

```
enum ENetRole
```

```
{
```

```
    ROLE_None,  
    ROLE_SimulatedProxy,  
    ROLE_AutonomousProxy,  
    ROLE_Authority,  
    ROLE_MAX,
```

```
};
```

```
void OnlineGameMode::EndSession()
```

```
{
```

```
    UnregisterEvents();
```

```
    if (ROLE_Authority != NM_DedicatedServer)
```

```
    {
```

```
        return;
```

```
    }
```

```
    TelemetryInterface.EndSession();
```

```
}
```

Code Review #4

```
enum ENetMode
```

```
{
```

```
    NM_StandAlone,  
    NM_DedicatedServer,  
    NM_ListenServer,  
    NM_Client,  
    NM_MAX,
```

```
};
```

```
enum ENetRole
```

```
{
```

```
    ROLE_None,  
    ROLE_SimulatedProxy,  
    ROLE_AutonomousProxy,  
    ROLE_Authority,  
    ROLE_MAX,
```

```
};
```

```
void OnlineGameMode::EndSession()
```

```
{
```

```
    UnregisterEvents();
```

```
    if (3 != 1)
```

```
    {
```

```
        return;
```

```
    }
```

```
    TelemetryInterface.EndSession();
```

```
}
```

C5054 / -Wdeprecated-enum-compare

Compiler warning (level 4) C5054



Summarize this article for me

operator '*operator-name*': deprecated between enumerations of different types

Remarks

C++20 has deprecated the usual arithmetic conversions on operands, where one operand is of enumeration type and the other is of a different enumeration type. For more information, see C++ Standard proposal [P1120R0](#).

In Visual Studio 2019 version 16.2 and later, an implicit conversion between enumeration types produces a level 4 warning when the `/std:c++latest` compiler option is enabled. In Visual Studio 2019 version 16.11 and later, it also produces a warning under `/std:c++20`.



Code Review #4 - Fix

```
void OnlineGameMode::EndSession()
{
    UnregisterEvents();

    if (GetRole() != NM_DedicatedServer)
    {
        return;
    }

    TelemetryInterface.EndSession();
}
```

Code Review #4 - Fix

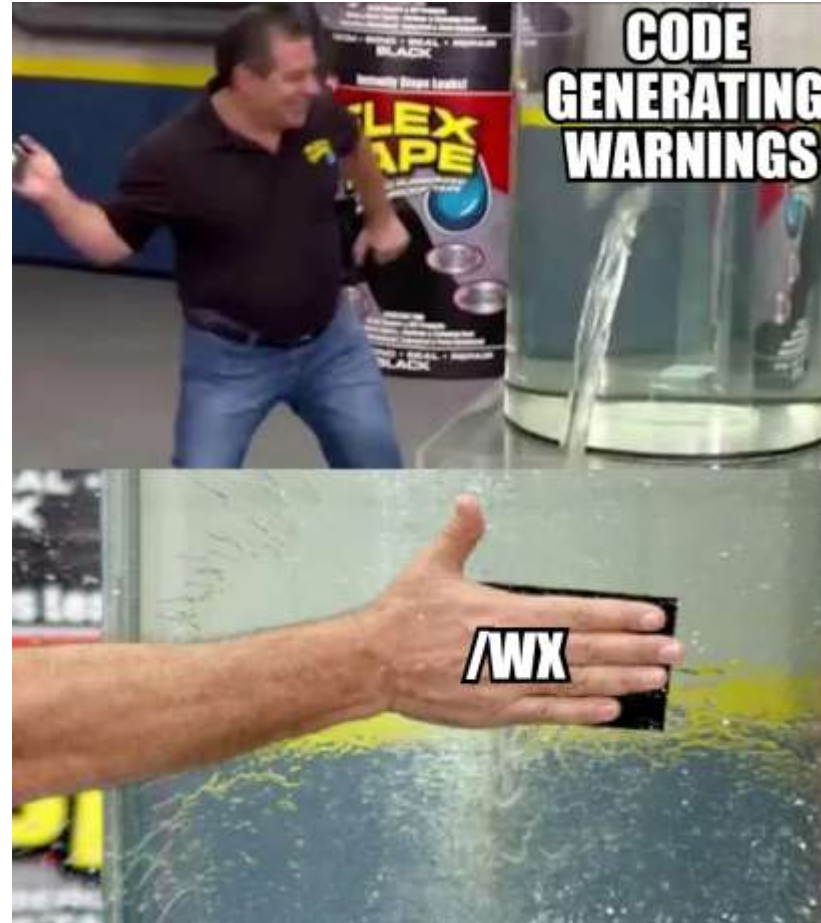
```
void OnlineGameMode::EndSession()  
{  
    UnregisterEvents();  
  
    TelemetryInterface.EndSession();  
}
```

... Are we done?

Build succeeded.
0 Warning(s)
0 Error(s)



Add /WX for Server



Are we really at /W4 /WX for Editor and Server?

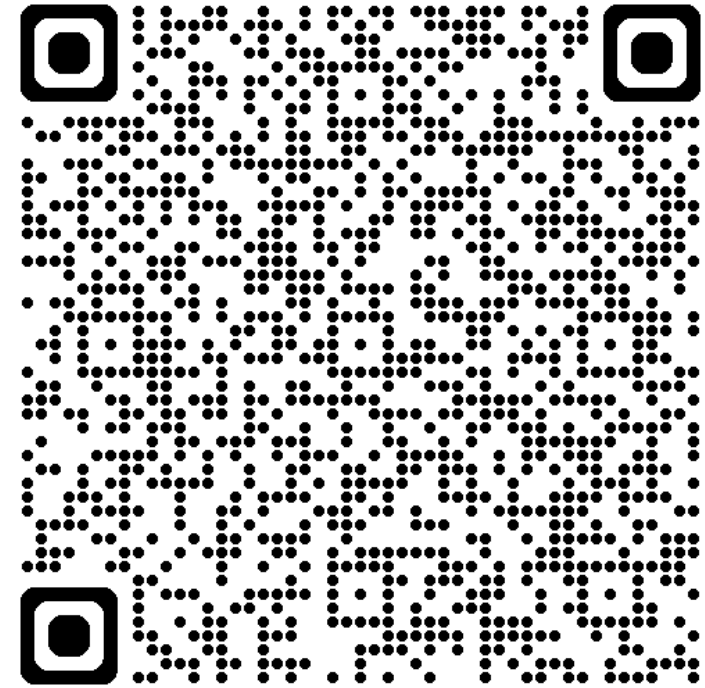


Enter WindowsPlatformCompilerSetup.h

```
134 // Unwanted VC++ level 4 warnings to disable.
135 #pragma warning(disable: 4100) // 'identifier' : unreferenced formal parameter
136 #pragma warning(disable: 4121) // 'symbol' : alignment of a member was sensitive to packing
137 #pragma warning(disable: 4127) // Conditional expression is constant
138 #pragma warning(disable: 4180) // qualifier applied to function type has no meaning; ignored
139 #pragma warning(disable: 4189) // 'identifier': local variable is initialized but not referenced
140
141 #pragma warning(disable: 4200) // Zero-length array item at end of structure, a VC-specific extension
142 #pragma warning(disable: 4201) // nonstandard extension used : nameless struct/union
143 #pragma warning(disable: 4217) // 'operator' : member template functions cannot be used for copy-assignment
144 #pragma warning(disable: 4245) // 'initializing': conversion from 'type' to 'type', signed/unsigned mismatch
145 #pragma warning(disable: 4251) // 'type' needs to have dll-interface to be used by clients of 'type'
146 #pragma warning(disable: 4267) // 'var' : conversion from 'size_t' to 'type', possible loss of data
147 #pragma warning(disable: 4275) // non - DLL-interface classkey 'identifier' used as base for DLL-interface
148 #pragma warning(disable: 4291) // typedef-name '' used as synonym for class-name ''
149
150 #pragma warning(disable: 4307) // '': integral constant overflow
151 #pragma warning(disable: 4315) // 'classname': 'this' pointer for member 'member' may not be aligned 'alignment'
152 #pragma warning(disable: 4316) // 'identifier': object allocated on the heap may not be aligned 'alignment'
153 #pragma warning(disable: 4324) // structure was padded due to __declspec(align())
154 #pragma warning(disable: 4347) // behavior change: 'function template' is called instead of 'function'
155 #pragma warning(disable: 4351) // new behavior: elements of array 'array' will be default initialized
156 #pragma warning(disable: 4355) // this used in base initializer list
157 #pragma warning(disable: 4366) // The result of the unary 'operator' operator may be unaligned
158 #pragma warning(disable: 4373) // '%%S': virtual function overrides '%%pS', previous versions of the compiler
159 #pragma warning(disable: 4389) // signed/unsigned mismatch
160
161 #pragma warning(disable: 4464) // relative include path contains '..'
162 #pragma warning(disable: 4482) // nonstandard extension used: enum 'enumeration' used in qualified name
...
```

Compare with latest Unreal Engine

```
131 // Unwanted VC++ level 4 warnings to disable.
132 #pragma warning(disable: 4100) // 'identifier' : unreferenced formal parameter
133 #pragma warning(disable: 4121) // 'symbol' : alignment of a member was sensitive to packing
134 #pragma warning(disable: 4127) // Conditional expression is constant
135 #pragma warning(disable: 4180) // qualifier applied to function type has no meaning; ignored
136 #pragma warning(disable: 4189) // 'identifier': local variable is initialized but not referenced
137
138 #pragma warning(disable: 4200) // Zero-length array item at end of structure, a VC-specific extension
139 #pragma warning(disable: 4201) // nonstandard extension used : nameless struct/union
140 #pragma warning(disable: 4217) // 'operator' : member template functions cannot be used for copy-assignment or copy-constr
141 #pragma warning(disable: 4245) // 'initializing': conversion from 'type' to 'type', signed/unsigned mismatch
142 #pragma warning(disable: 4251) // 'type' needs to have dll-interface to be used by clients of 'type'
143 #pragma warning(disable: 4267) // 'var' : conversion from 'size_t' to 'type', possible loss of data
144 #pragma warning(disable: 4275) // non - DLL-interface classkey 'identifier' used as base for DLL-interface classkey 'ident
145 #pragma warning(disable: 4291) // no matching operator delete found; memory will not be freed if initialization throws an
146
147 #pragma warning(disable: 4307) // '': integral constant overflow
148 #pragma warning(disable: 4315) // 'classname': 'this' pointer for member 'member' may not be aligned 'alignment' as expect
149 #pragma warning(disable: 4316) // 'identifier': object allocated on the heap may not be aligned 'alignment'
150 #pragma warning(disable: 4324) // structure was padded due to __declspec(align())
151 #pragma warning(disable: 4347) // behavior change: 'function template' is called instead of 'function
152 #pragma warning(disable: 4351) // new behavior: elements of array 'array' will be default initialized
153 #pragma warning(disable: 4355) // this used in base initializer list
154 #pragma warning(disable: 4366) // The result of the unary 'operator' operator may be unaligned
155 #pragma warning(disable: 4373) // '%$S': virtual function overrides '%$pS', previous versions of the compiler did not over
156 #pragma warning(disable: 4389) // signed/unsigned mismatch
157
158 #pragma warning(disable: 4464) // relative include path contains '..'
159 #pragma warning(disable: 4482) // nonstandard extension used: enum 'enumeration' used in qualified name
```

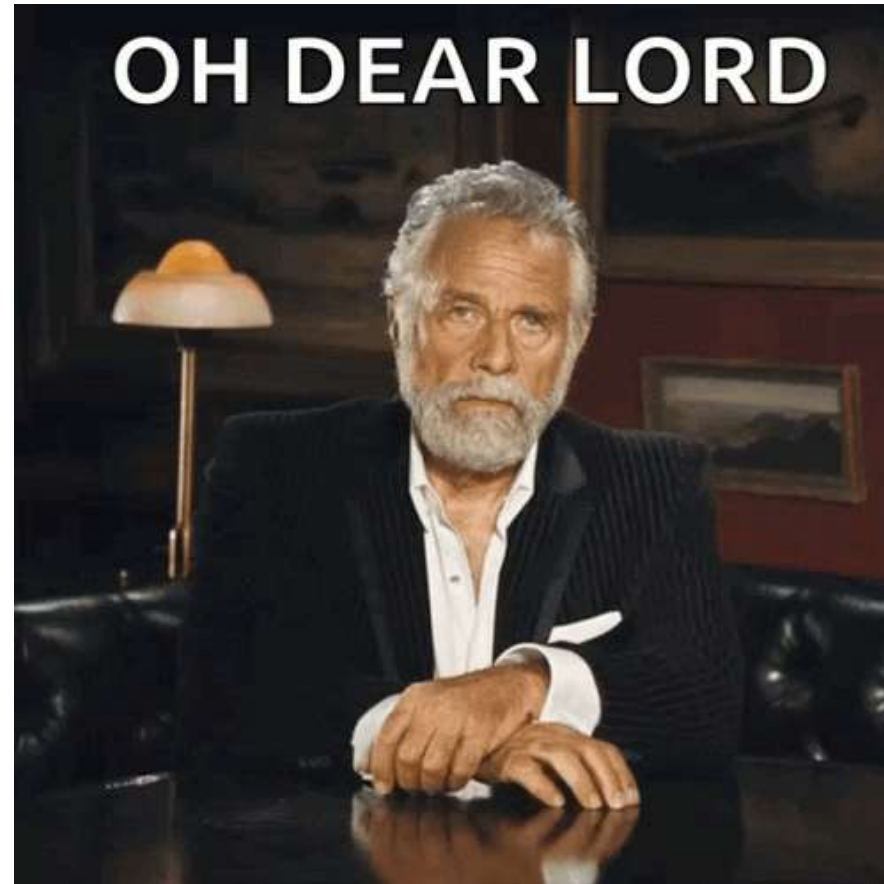


This affects every Unreal Engine game



What about the response files?

/wd4463
/wd4838
/wd4458
/wd4456
/wd4459
/wd4819



New Plan!



New Plan!

```
editor.Disable(warningsAsErrors);

for (auto& warning : disabledWarnings)
{
    warning.Enable();
    const auto buildResult = editor.Build();

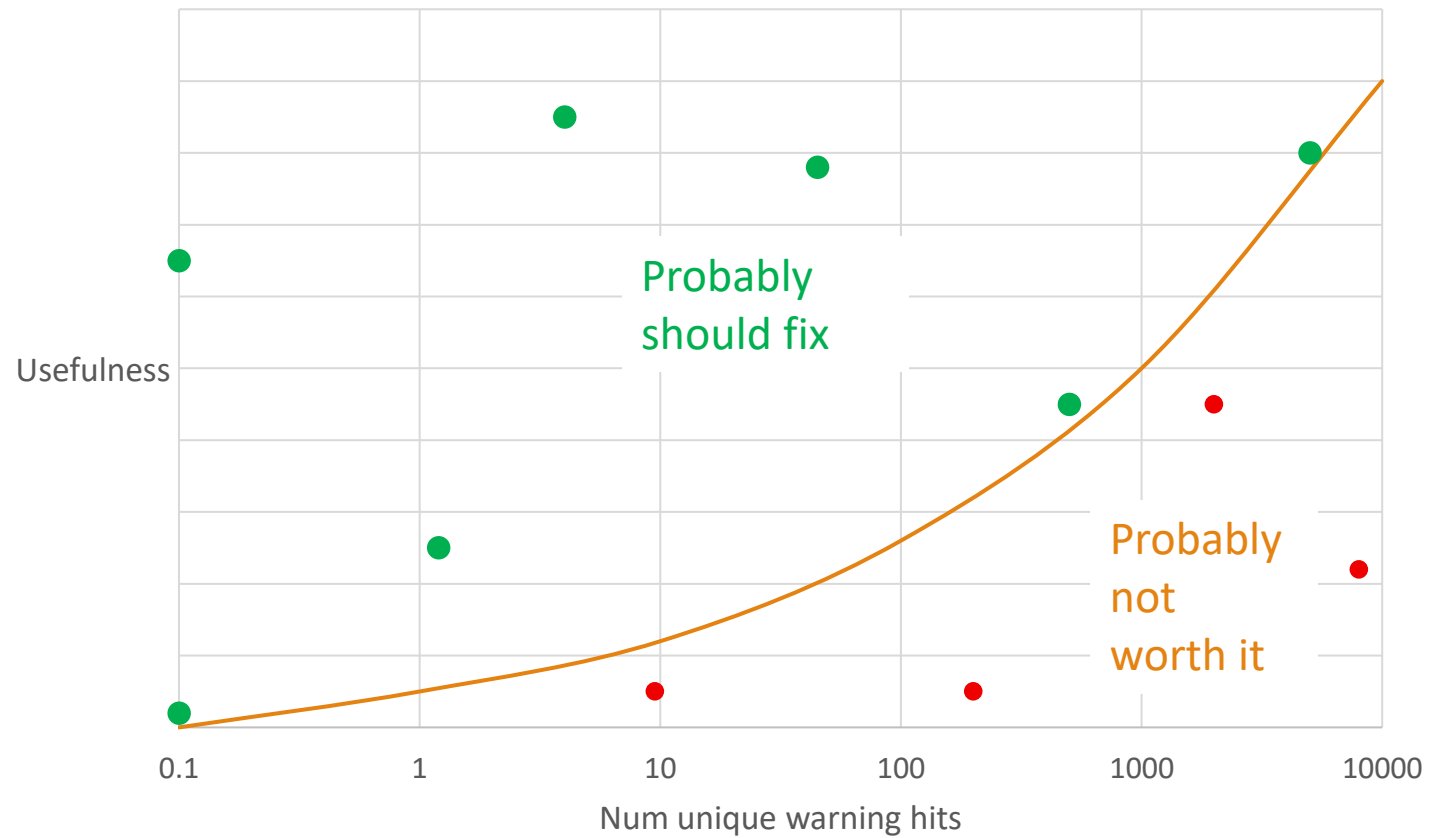
    if (buildResult.Success())
    {
        pendingWarningsToEnable.Add(warning);
    }
}
```

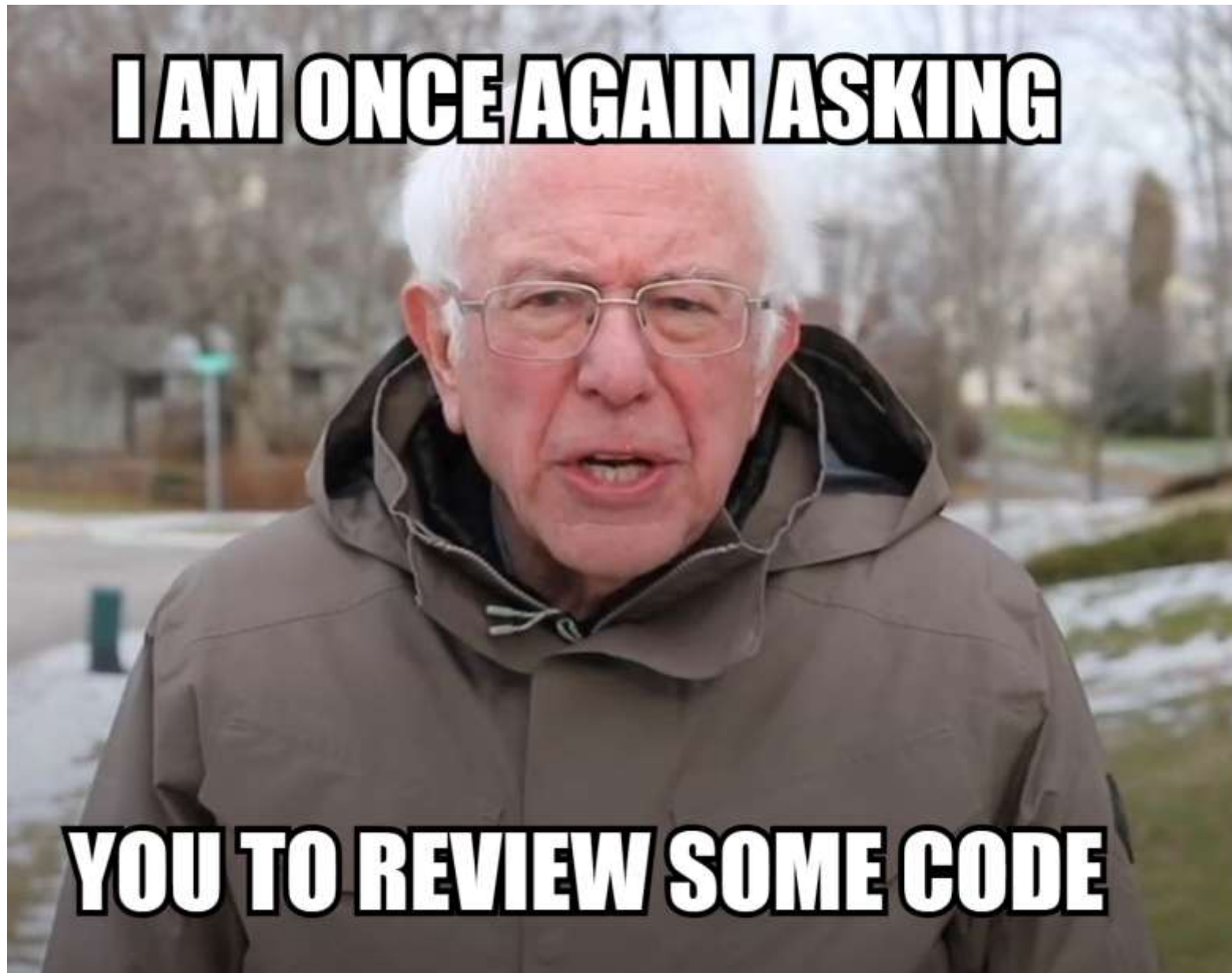
New Plan!

```
editor.Disable(warningsAsErrors);

for (auto& warning : disabledWarnings)
{
    else
    {
        if (ShouldFix(warning, buildResult))
        {
            Fix(warning);
            pendingWarningsToEnable.Add(warning);
        }
        else
        {
            warning.disable();
        }
    }
}
editor.Enable(warningsAsErrors);
```

Heuristics for ShouldFix()





Code Review #5

```
// CoreLib.cpp
static void DoSomethingHelper()
{
    // Many lines of complicated code.
}

void CoreLib::DoSomething()
{
    DoSomethingHelper();
    // Some more stuff
}
```

Code Review #5

```
// CoreLib.cpp
static void DoSomethingHelper()
{
    // Many lines of complicated code.
}

void CoreLib::DoSomething()
{
    // Some more stuff
}
```

C4505 / -Wunused-function

Compiler Warning (level 4) C4505



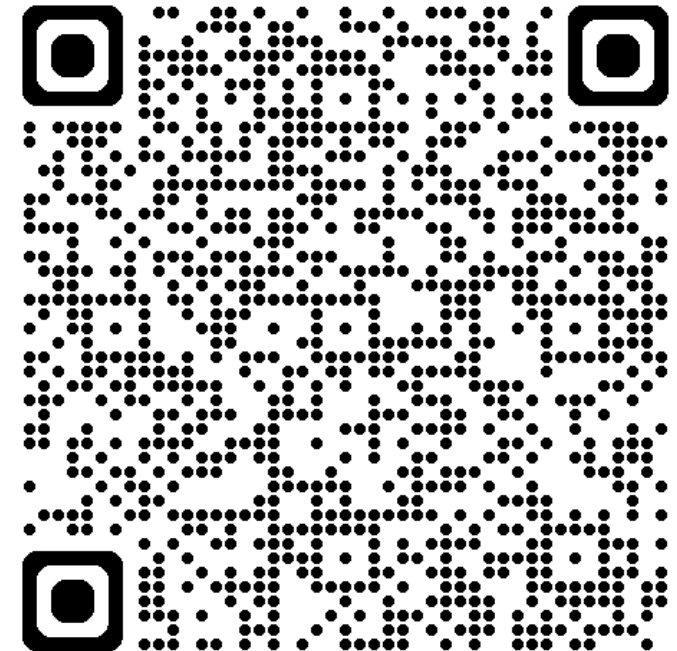
Summarize this article for me

'function' : unreferenced local function has been removed

Remarks

The given function is local and not referenced in the body of the module; therefore, the function is dead code.

The compiler did not generate code for this dead function.



Code Review #5

```
// CoreLib.cpp
static void DoSomethingHelper()
{
    // Many lines of complicated code.
}
```

```
void CoreLib::DoSomething()
{
    DoSomethingHelper();
    // Some more stuff
}
```

Code Review #5 - Fix

```
// CoreLib.cpp
```

```
void CoreLib::DoSomething()  
{  
    // Some more stuff  
}
```

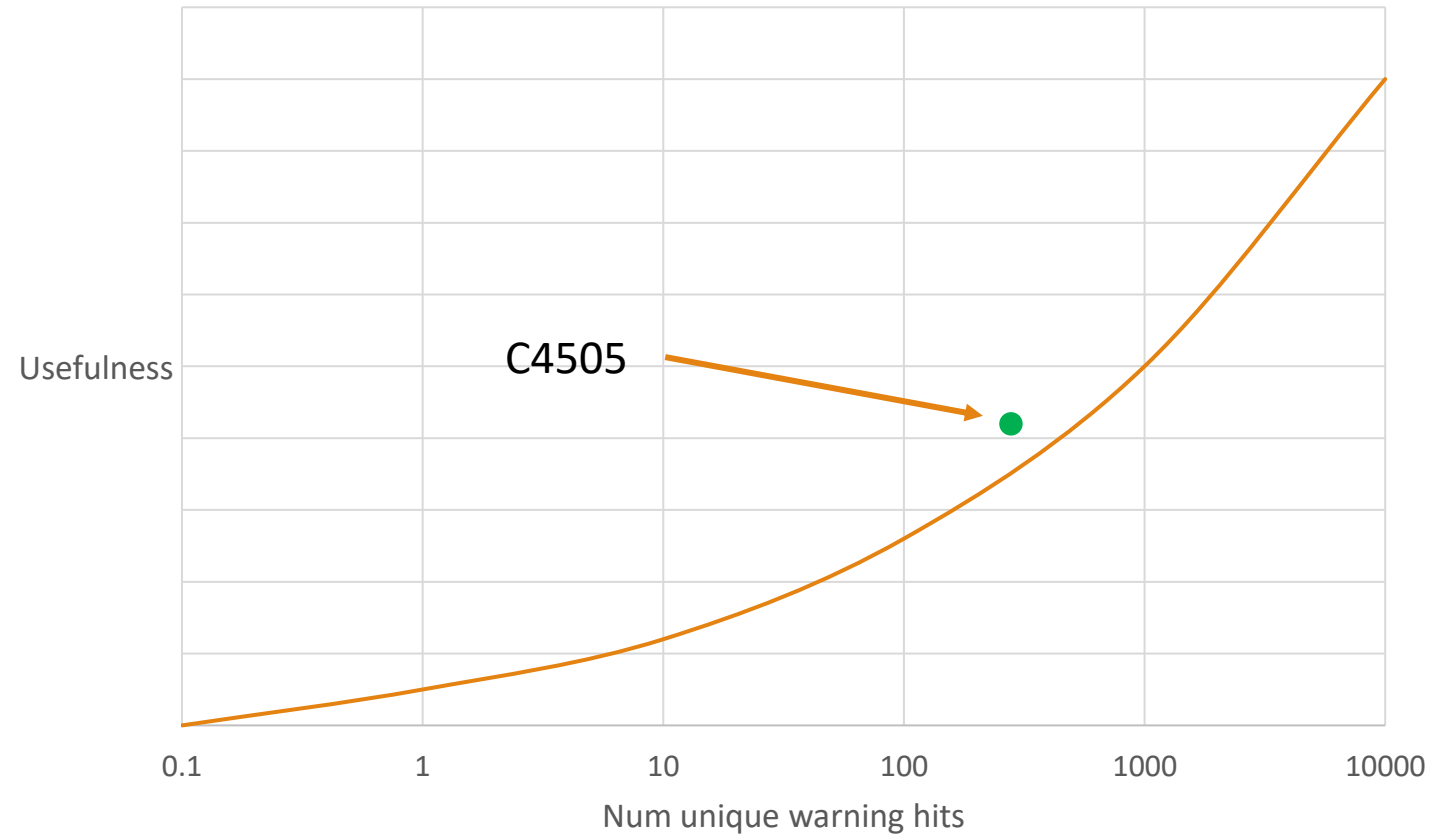
Code Review #5.1

```
// CoreLib.h
static void DoSomethingHelper()
{
    // Many lines of complicated code.
}
```

Code Review #5.1 - Fix

```
// CoreLib.h
inline void DoSomethingHelper()
{
    // Many lines of complicated code.
}
```

Should we fix C4505?



Any binary size savings?

1KB!

Not really...

1KB!

 `class AActor`

- * Actor is the base class for an Object that can be placed or spawned in a level.
- * Actors may contain a collection of ActorComponents, which can be used to control how actors move, how they are rendered, etc.
- * The other main function of an Actor is the replication of properties and function calls across the network during play.
- * @see <https://docs.unrealengine.com/latest/INT/Programming/UnrealArchitecture/Actors/>
- * @see UActorComponent

 [Describe](#)

Size: 1096 bytes

Alignment: 8 bytes

Not really...

1KB!

 `class AActor`

* Actor is the base class for an Object that can be placed or spawned in a level.
* Actors may contain a collection of

Size: 1096 bytes

Alignment: 8 bytes

Programming/UnrealArchitecture/Actors/
* @see UActorComponent

 [Describe](#)

Size: 1096 bytes

Alignment: 8 bytes

Code Review #6

```
class A
{
public:
    ~A();
    virtual void DoSomething();
};
```

Code Review #6 - Fix

```
class A
{
public:
    virtual ~A();
    virtual void DoSomething();
};
```

C4265/-Wdelete-non-virtual-dtor

Compiler Warning (level 3, off) C4265



Summarize this article for me

'*classname*': class has virtual functions, but its non-trivial destructor is not virtual; instances of this class may not be destructed correctly

Remarks

When a class has virtual functions but a nonvirtual destructor, objects of the type might not be destroyed properly when the class is destroyed through a base class pointer.

This warning is off by default. For more information, see [Compiler Warnings That Are Off by Default](#).



Old New Thing on C4265

June 18th, 2020

👍 1 reaction

Understanding warning C4265: class has virtual functions, but destructor is not virtual, part 1

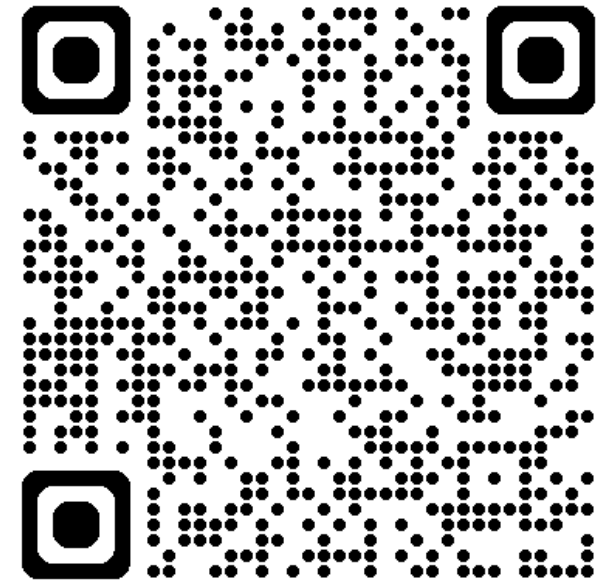


Raymond Chen

A few customers have noted that compiling WRL and C++/WinRT headers generate warning C4265: class has virtual functions, but destructor is not virtual.

What's this warning trying to say?

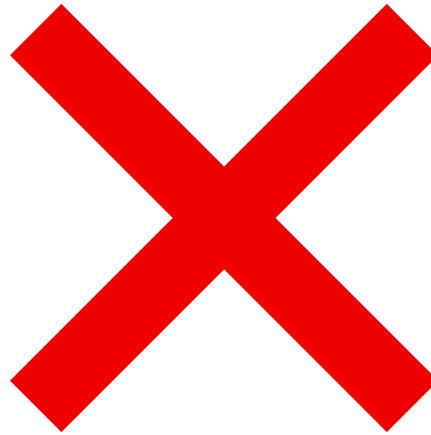
If a non-final class has a virtual method, then the compiler considers the possibility that there will be a derived class which overrides that virtual method. And if the class does not have a virtual destructor when a pointer to the base class is `delete`d, then only the destructor for the base class will run; the destructor for the derived class will not run.¹



How much of an issue is this?

```
struct A
{
    ~A() = default;
};
```

```
struct B : A
{
    ~B()
    {
        std::puts("Hello CppCon!");
    }
};
```



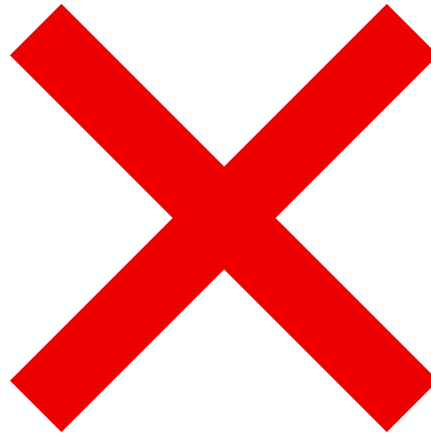
```
A* GetRaw()
{
    return new B;
}
```

```
int main()
{
    auto result = GetRaw();
    delete result;
}
```

How much of an issue is this?

```
struct A
{
    ~A() = default;
};
```

```
struct B : A
{
    ~B()
    {
        std::puts("Hello CppCon!");
    }
};
```



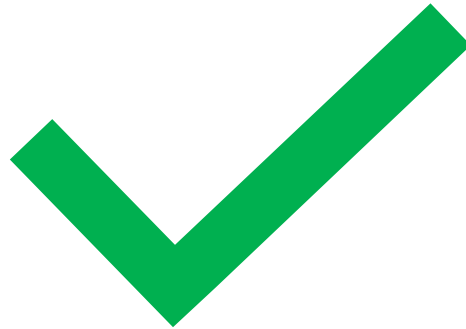
```
std::unique_ptr<A> GetUnique()
{
    return std::make_unique<B>();
}
```

```
int main()
{
    auto result = GetUnique();
}
```

How much of an issue is this?

```
struct A
{
    ~A() = default;
};
```

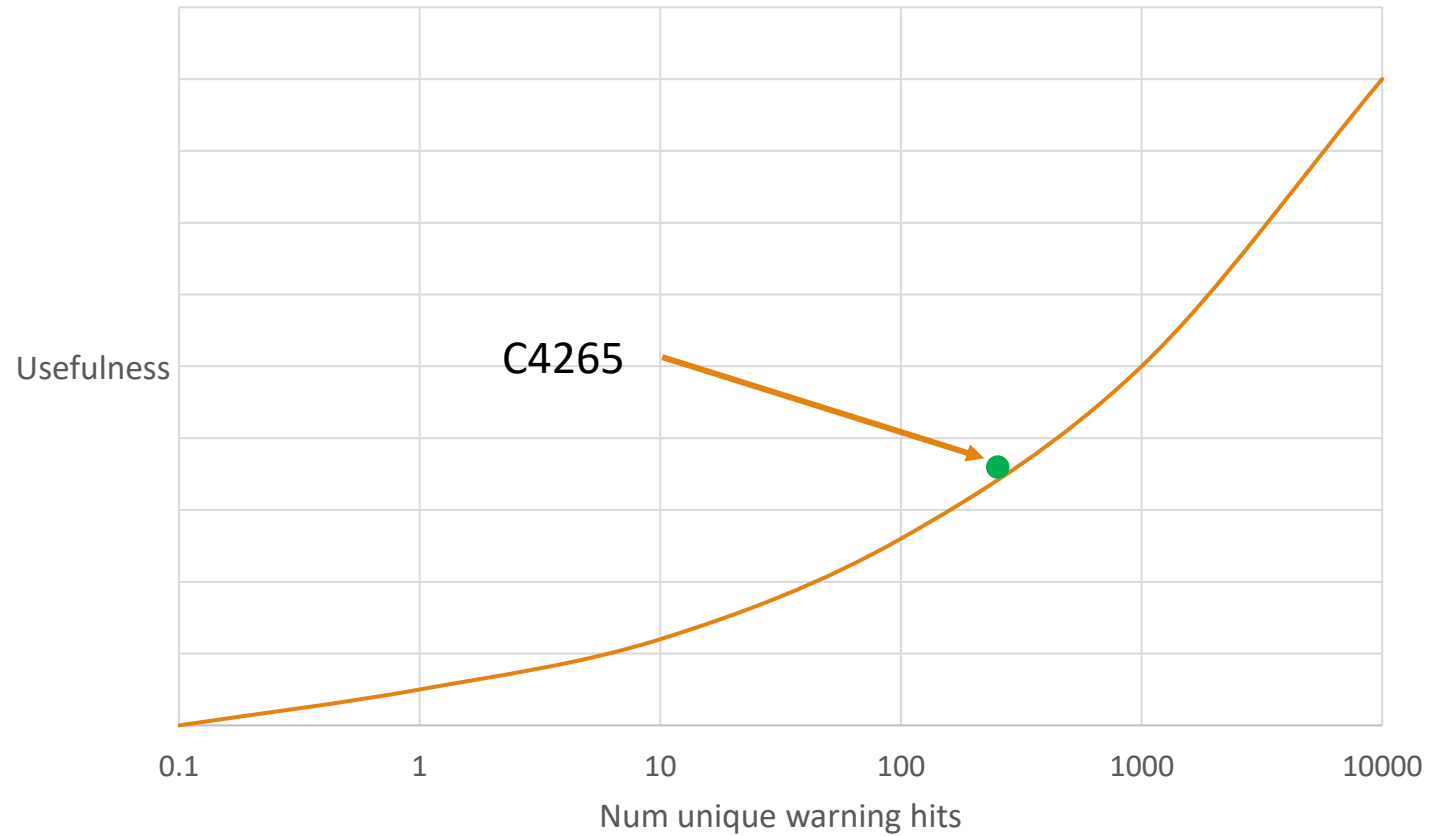
```
struct B : A
{
    ~B()
    {
        std::puts("Hello CppCon!");
    }
};
```



```
std::shared_ptr<A> GetShared()
{
    return std::make_shared<B>();
}
```

```
int main()
{
    auto result = GetShared();
}
```

Should we fix C4265?



Code Review #7

```
struct A
{
    uint32 Field = 0;
};

int32 Do(const A& InA, const int32 Test)
{
    if (InA.Field != Test)
    {
        return 1;
    }

    return 0;
}
```

C4389 / Wsign-compare

Compiler Warning (level 4) C4389



Summarize this article for me

'equality-operator' : signed/unsigned mismatch

Remarks

An `==` or `!=` operation involved `signed` and `unsigned` variables. This could result in a loss of data.

One way to fix this warning is if you cast one of the two types when you compare `signed` and `unsigned` types.



Well-known good practice

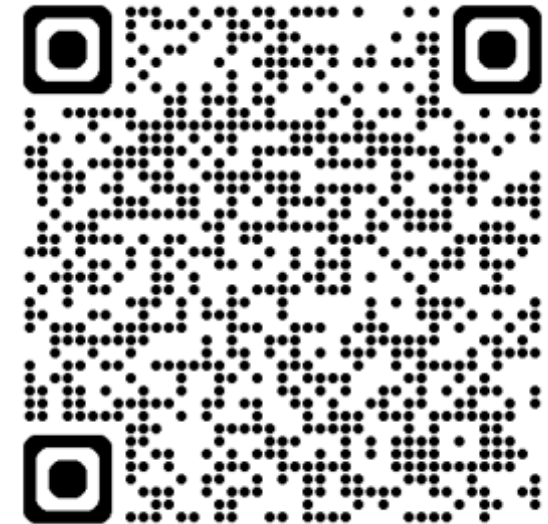
My battle against signed/unsigned comparison: the worst violations

Sandor Dargo on Oct 24, 2023 • 7 min read

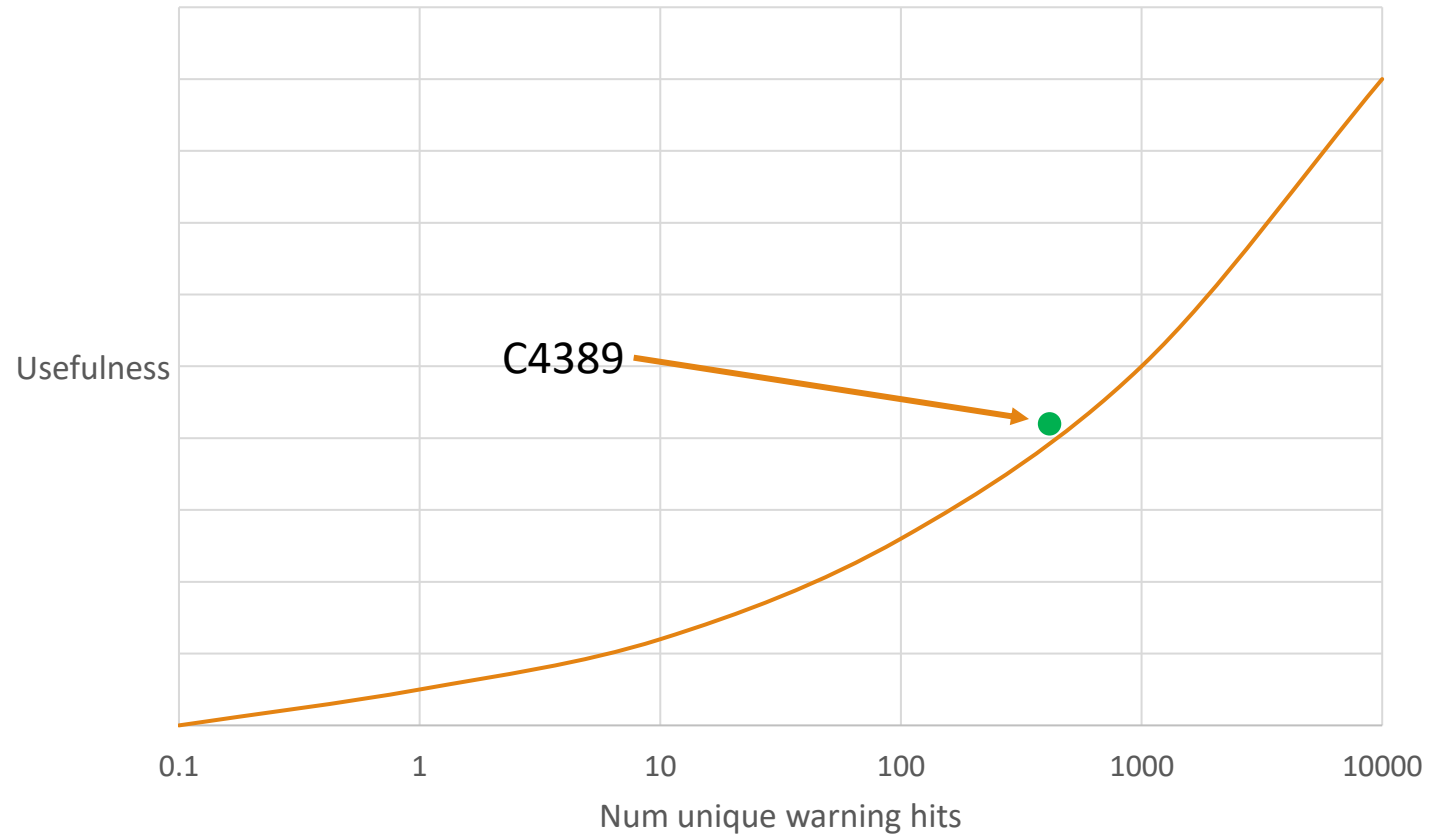
We spent the last two weeks discussing the dangers of signed/unsigned comparisons. First, we discussed why it's a problem in the first place, and [how we can safely compare signed and unsigned types to each other](#), then I shared with you which are [the most common ways -Wsign-compare is violated](#).

This week, let me share with you the strangest violations. The worst offenders, the things I didn't want to see.

(Obviously I simplified and anonymized the examples)



Should we fix C4389?



Code Review #7.1

```
struct A
{
    uint32 Field = 0;
};

int32 Do(const A& InA, const int32 Test)
{
    if (InA.Field != Test)
    {
        return 1;
    }

    return 0;
}
```

Code Review #7.1

```
struct A
{
    uint32 Field : 1 = false;
};

int32 Do(const A& InA, const bool Test)
{
    if (InA.Field != Test)
    {
        return 1;
    }

    return 0;
}
```

Code Review #7.1

```
struct A
{
    uint32 Field : 1 = false;
};

int32 Do(const A& InA)
{
    if (InA.Field != false)
    {
        return 1;
    }

    return 0;
}
```

See a bug, report a bug

 **C4389 is triggered when comparing a bitfield member of one bit with a bool**

New 0 0 0

 0
Votes

 **Keith Stockdale [MSFT]** - Reported Mar 20, 2026 7:04 AM [View internal ticket](#)

[severity:It bothers me. A fix would be nice]
Hi!

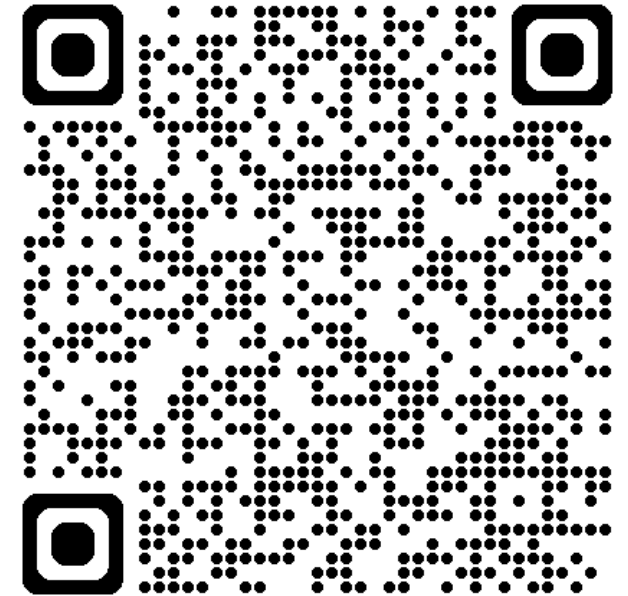
Given the following code (which can also be found at <https://godbolt.org/z/GcKnjbhMn>)

```
struct A
{
    unsigned int Field:1 = false;
};

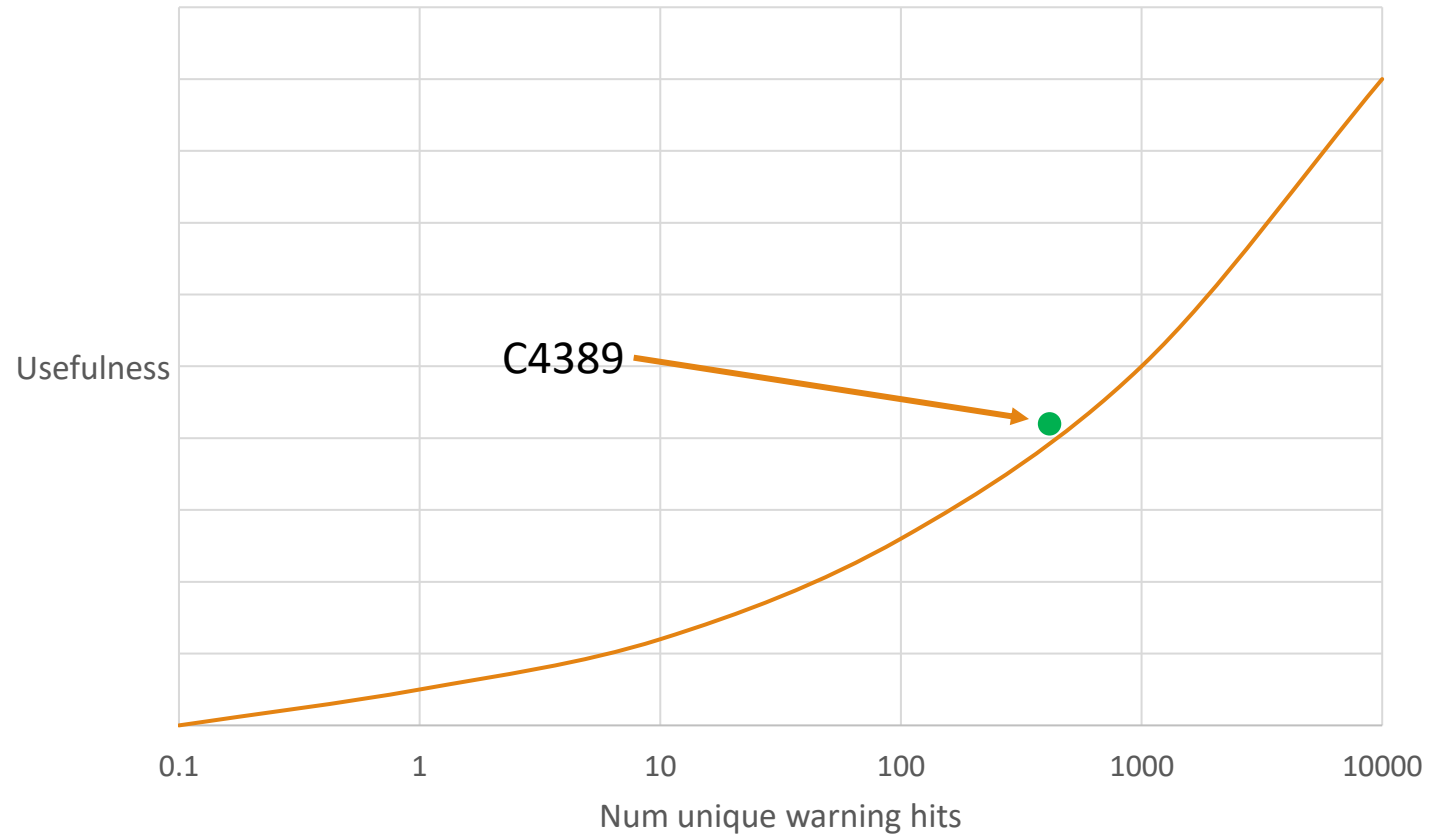
int Thing(A InA, const bool Test)
{
    if (InA.Field != Test)
    {
        return 1;
    }

    return 0;
}
```

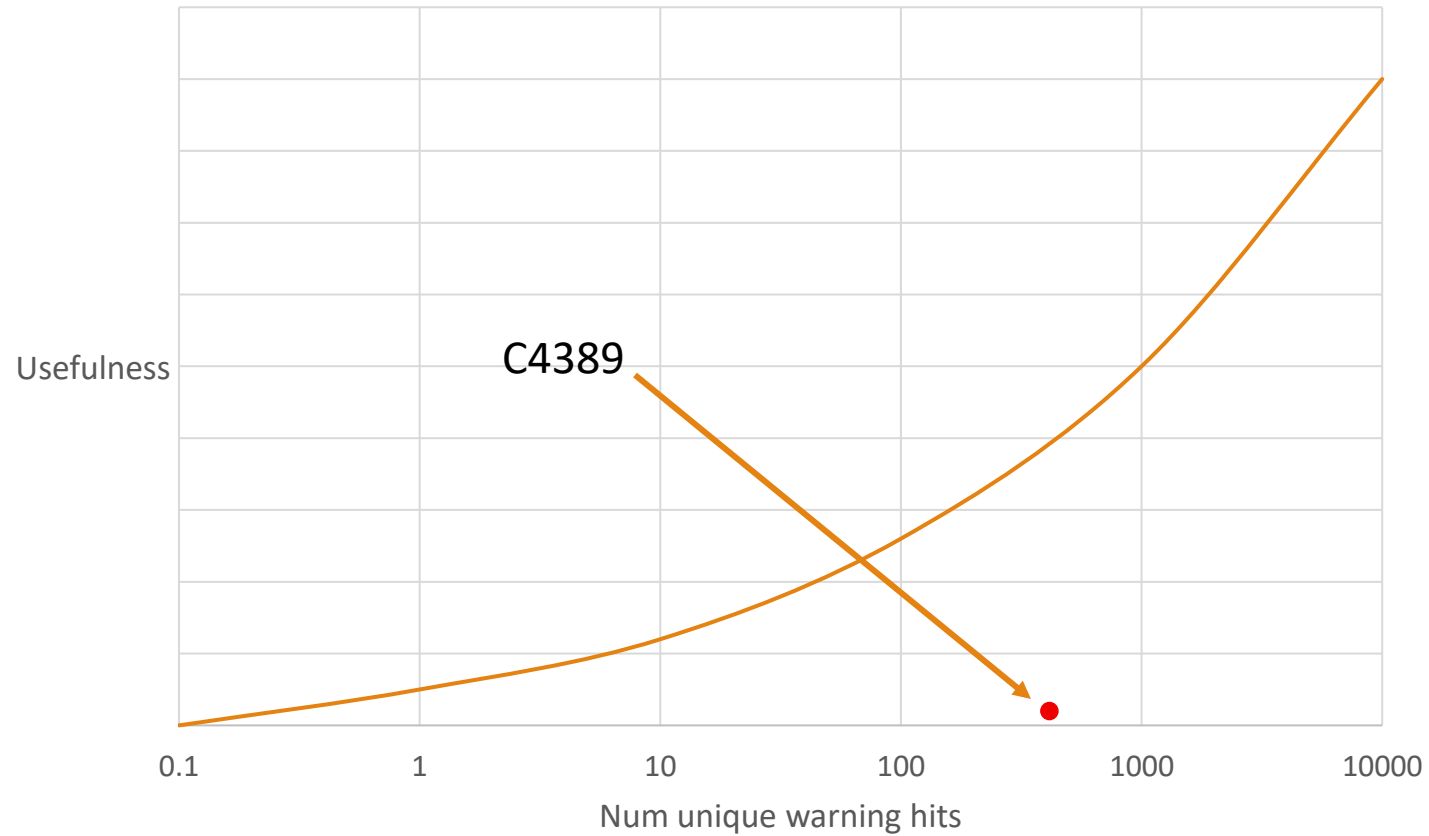
Warning C4389 will be emitted when comparing `InA.Field != Test`. I believe this to be erroneous for a couple of reasons.



Should we fix C4389?



Should we fix C4389?



... But maybe later

 **C4389 is triggered when comparing a bitfield member of one bit with a bool** Closed - Fixed View Solution 1 6 ...

 **3**
 Votes

 **Keith Stockdale [MSFT]** · Reported Mar 20, 2026 7:04 AM · [View internal ticket C4389](#)

[severity:It bothers me. A fix would be nice]
Hi!

Given the following code (which can also be found at <https://godbolt.org/z/GcKnjbhMn>)

```
struct A
{
    unsigned int Field:1 = false;
};

int Thing(A InA, const bool Test)
{
    if (InA.Field != Test)
    {
        return 1;
    }

    return 0;
}
```

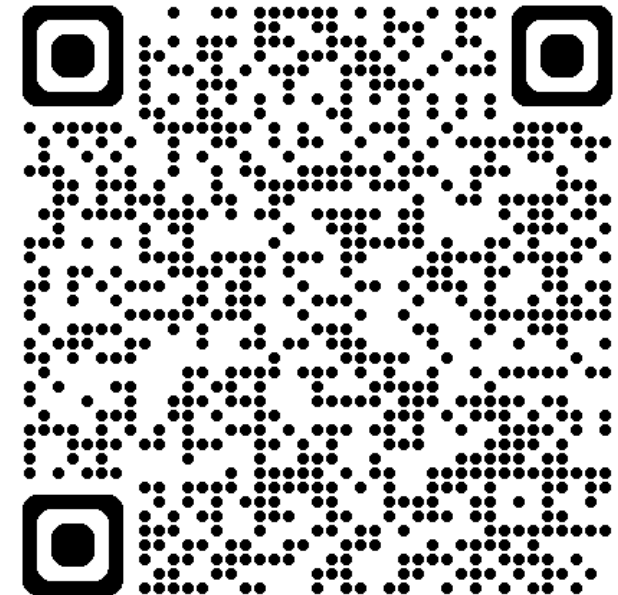
Warning C4389 will be emitted when comparing `InA.Field != Test`. I believe this to be erroneous for a couple of reasons.

1. Clang and GCC don't have an issue with it when they have `-Wsign-compare` enabled
2. C4389 is not emitted when comparing the bit field with a constant expression e.g. <https://godbolt.org/z/TTPea6jG>

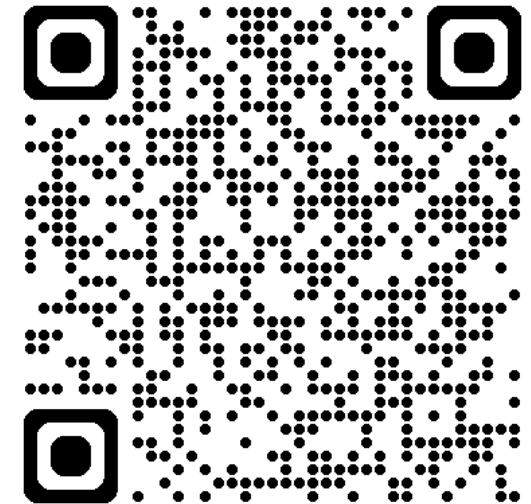
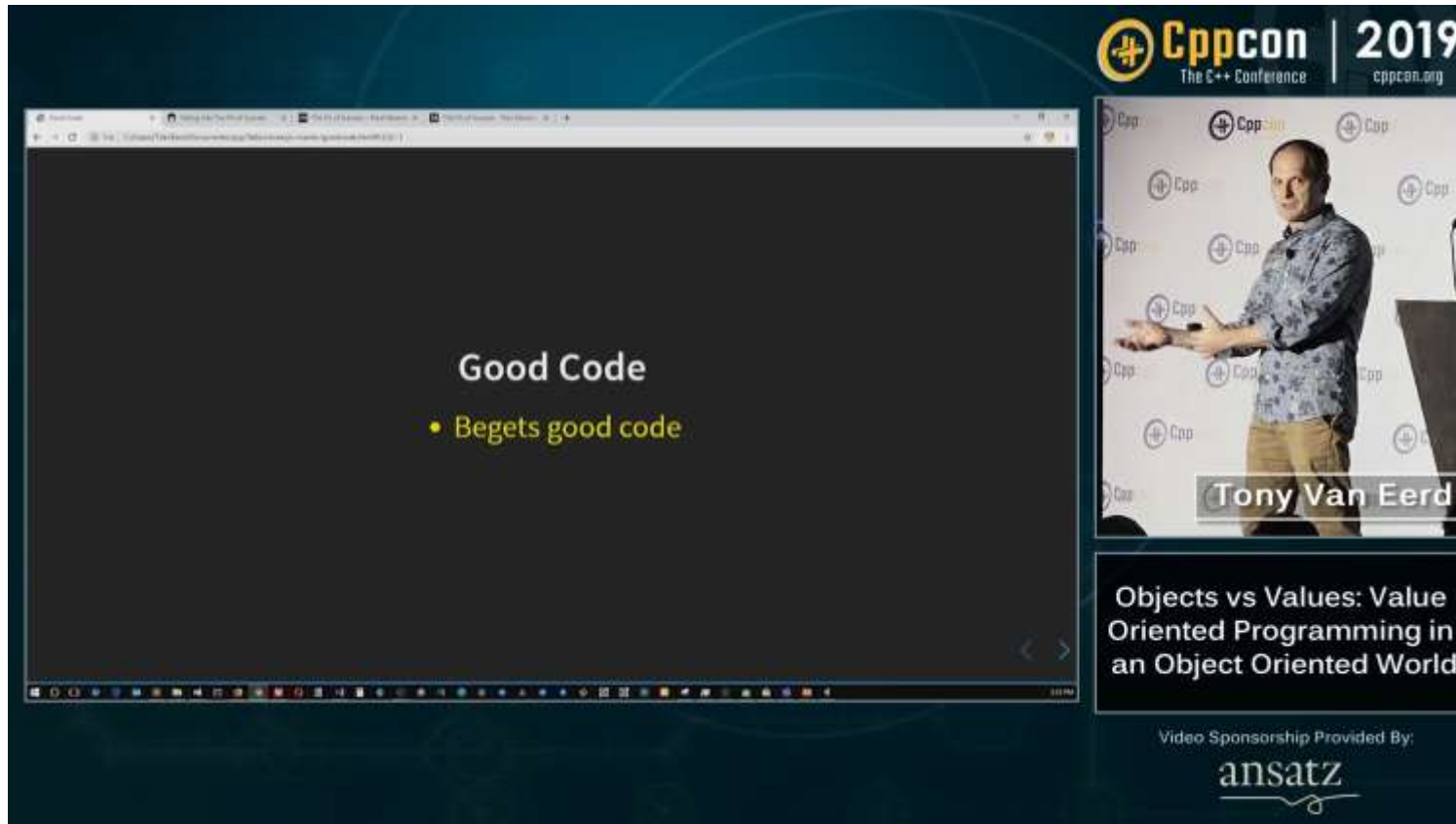
This has come up as I want to enable this warning in our codebase but can't because we make heavy use of bitfields.

Hopefully this is enough information to solve the problem,
Thanks,
Keith

Fixed In: 18.6.0 Visual Studio windows 10.0 category:cpp Visual Studio 18.5 Insiders 1 



“Good code begets good code”



Code Review #8

```
const FHLslToken* FHLslScanner::GetCurrentTokenAndAdvance()
{
    if (CurrentToken < Tokens.Num())
    {
        auto* Return = &Tokens[CurrentToken];
        Advance();
    }

    return nullptr;
}
```

Code Review #8 - Fix

```
const FHLslToken* FHLslScanner::GetCurrentTokenAndAdvance()
{
    if (CurrentToken < Tokens.Num())
    {
        auto* Return = &Tokens[CurrentToken];
        Advance();
        return Return;
    }

    return nullptr;
}
```

C4189 / -Wunused-variable

Compiler Warning (level 3 and level 4) C4189



Summarize this article for me

'identifier' : local variable is initialized but not referenced

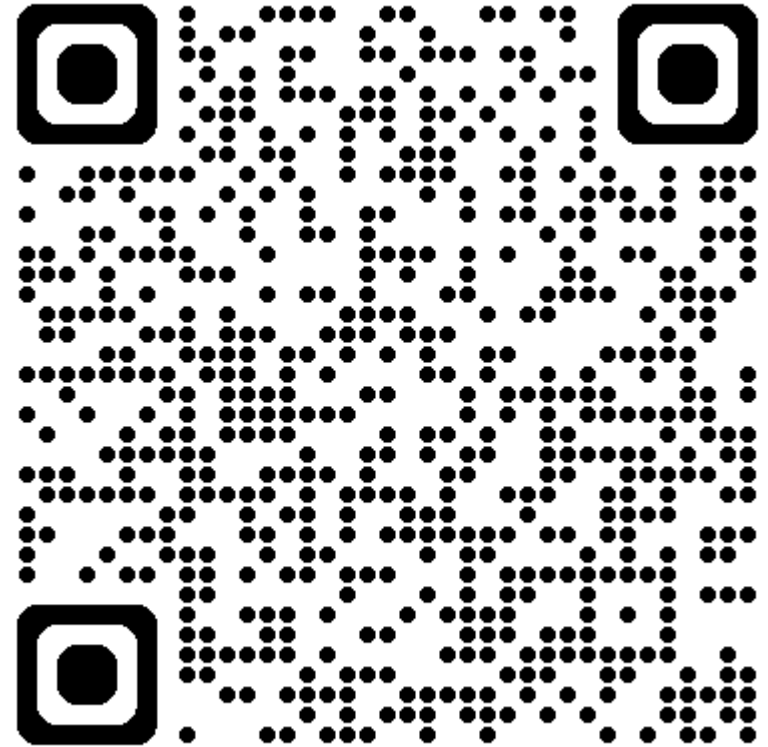
Remarks

A variable is declared and initialized but not used.



C4189 does not catch this...

```
struct A {};  
  
int main()  
{  
    A a{};  
}
```



... and people have reported it

 **Compiler Warning for Unreferenced Variables similar to Clang**

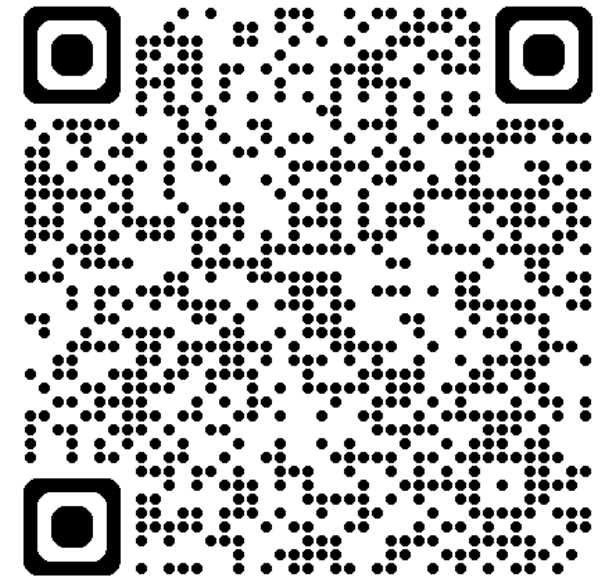
Under Review 11  ...

 **24**
 Votes

 **TS** [Taylor Saunders \[MSFT\]](#) - Reported May 09, 2019 6:22 PM

Clang has warnings for "unused-variable" and "unused-const-variable" that MSVC does not seem to have. C4101 and C4189 will catch non-static, non-const local variables, but there are no warnings that will catch any form of unused global variables or static/const local variables.

C++ `cppcompiler`



Code Review #8.1

```
struct FAllocTask
{
    static void Alloc()
    {
        int* IntAlloc = new int[1000];
        int8* LeakTask = new int8[1000000];
        SAFEDELETEARRAY( IntAlloc);
    }
};
```

Code Review #8.2

```
void DoSomething()
{
    auto result
        = ExpensiveFunctionWithNoSideEffects();
    // Never use result :)
}
```

Code Review #8.2

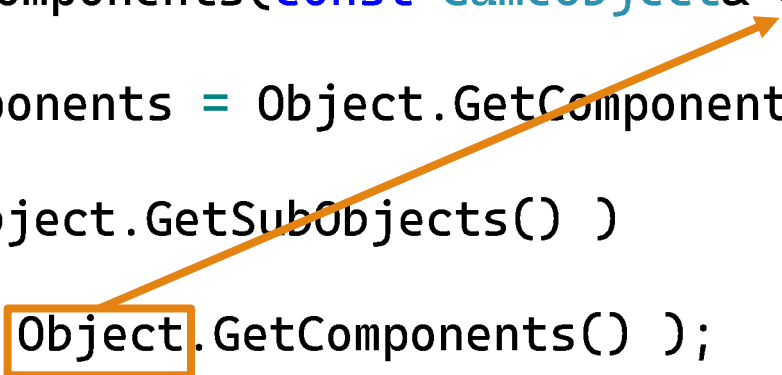
```
void DoSomething()  
{  
    // Much quicker ;)  
}
```

Code Review #8.3

```
TArray<Component> GetAllComponents(const GameObject& Object)
{
    TArray<Component> Components = Object.GetComponents();

    for ( auto& Object : Object.GetSubObjects() )
    {
        Components.Append( Object.GetComponents() );
    }

    return Components;
}
```



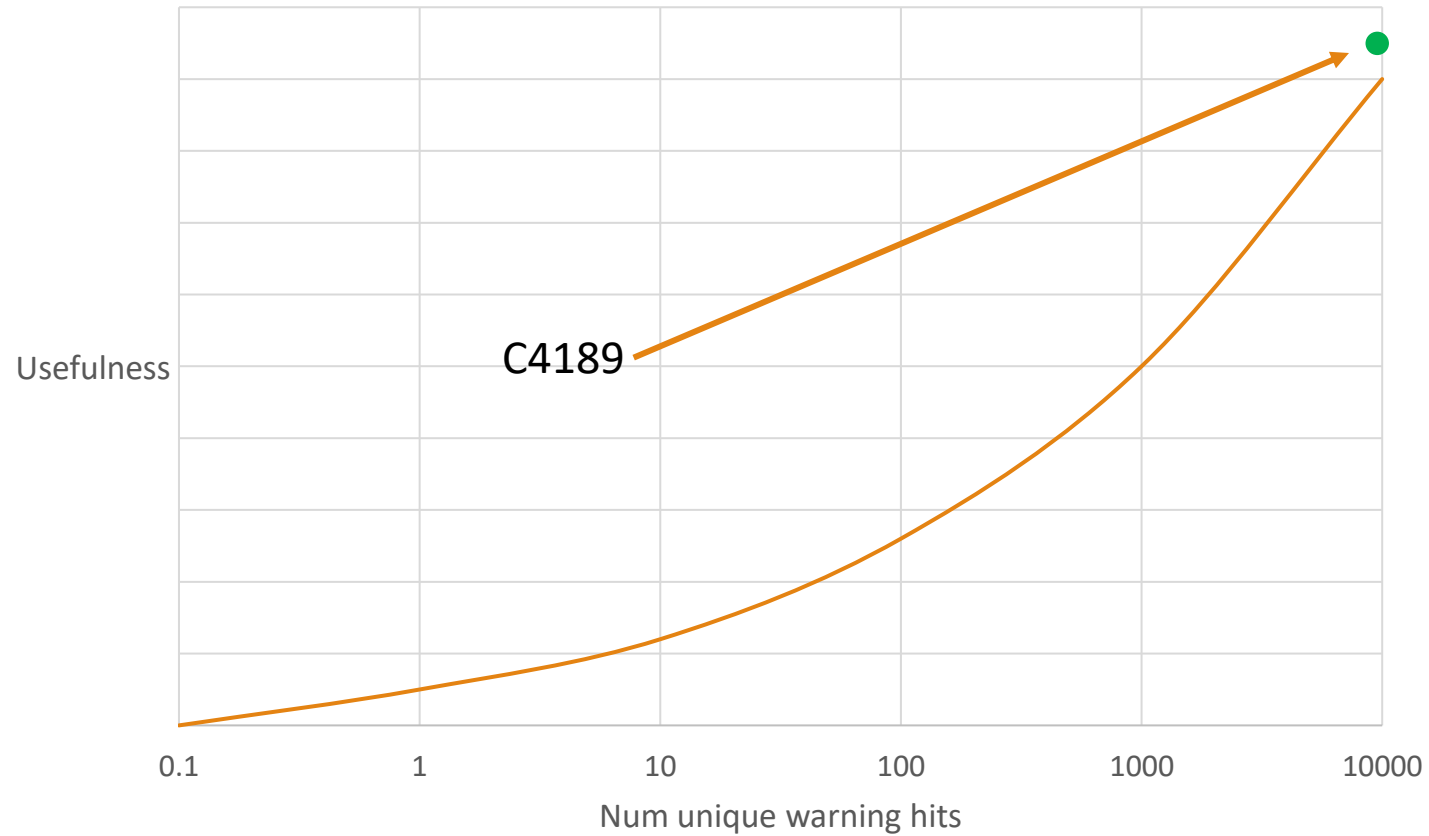
Code Review #8.3

```
TArray<Component> GetAllComponents(const GameObject& Object)
{
    TArray<Component> Components = Object.GetComponents();

    for ( auto& SubObject : Object.GetSubObjects() )
    {
        Components.Append( SubObject.GetComponents() );
    }

    return Components;
}
```

Should we fix C4189?





C4189 is the
final boss

Current Progress on C4189

NUMBER OF UNIQUE WARNINGS

At the start	9525
At BeC++ 2026 (March)	4977
At ACCU on Sea 2026 (June)	2754
At C++Con 2026 (September)	0

Thanks to all my reviewers

- Abhishek Chakraborty
- Becky Morley
- Benjamin Bloc
- Chantelle Porritt
- Charlotte Cleary
- Christopher Johnson
- Colin Gulliver
- David Dempsey
- David Meen
- Duncan Rendall
- Ed Ainsley
- Ian Norris
- Jon Holmes
- Josef Gluyas
- Kyle Sheridan
- Kyle Thorburn
- Mark Lucas
- Mathew Gregory
- Mattias Refeyton
- Maxime Piquot
- Mikael Enarsson
- Nadia Nadeem
- Neil Lindsey
- Quentin Warnant
- Riccardo Battello
- Sam Collier
- Sarah Ryan
- Stuart Holland
- Zoe Lee

They kept me honest



Line 1056

Quentin Warnant on 2026-07-17 at 16:51 ([4th upload](#))



Created issue D187659:
this looks like a behaviour change

Keith Stockdale on 2026-07-17 at 16:54 ([latest upload](#))



** Accepted * @1494910*

Quentin Warnant on 2026-07-21 at 10:01 ([4th upload](#))



Marked issue fixed: D187659

No “fixing” things



An example reaction when people heard about this work

[Fix C4189 - unreferenced local variable...](#)

Essentially I am on a crusade to re-enable this warning. And to do that I am fixing every single site of it SoT. Which basically amounts to deleting a load of code that does nothing or is not needed. 😊

10 June

David Dempsey 10/06 08:45

You're doing God's work Keith

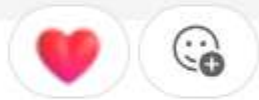
Is there a God of code? If there isn't we should make one

I'm gonna call him Barry which is short for Binary

Attitude towards the reviews

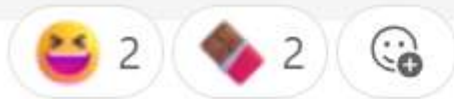
Mattias Refeyton 10/07 13:57

I'm happy to do a lot of these in a row, put on a podcast, get through all of them 🙌



Quentin Warnant (He/Him) 10/07 13:57

I expect 1 kitkat for every one of these reviews I processed



What about current day Unreal Engine?

Lyra Sample Game

Explore how to develop projects in UE5 using techniques from the Lyra Sample Game.



Lyra is a learning resource designed as a sample game project to help you understand the frameworks of Unreal Engine 5 (UE5). Its architecture is designed to be modular, including a core system and plugins that are updated regularly along with the development of UE5.

What about current day Unreal Engine?

NUMBER OF UNIQUE WARNINGS

Lyra

19

Engine

6248

Wrapping Up

So how hard is it?

Deliverable	Number of diffs	Number of Reviewed CLs
/WX (Editor)	105	9
/WX (Server)	2	2
Enable C4121	2	1
Enable C4373	2	1
Enable C4180	11	2
Enable C4091	28	4
Enable C4505	280	11
Enable C4265	254	18
Enable C4189	7628	154

Warnings are USEFUL...



They find bugs

Caught at compile time, long before tests, reviewers, QA or even players see them



They find dead code

Unused variables, unreachable branches and unused functions instantly



They find perf issues

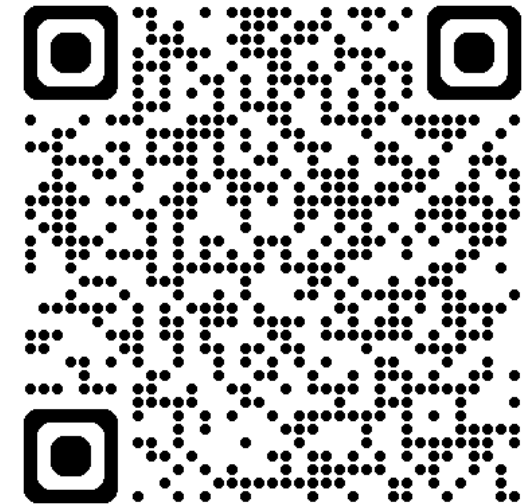
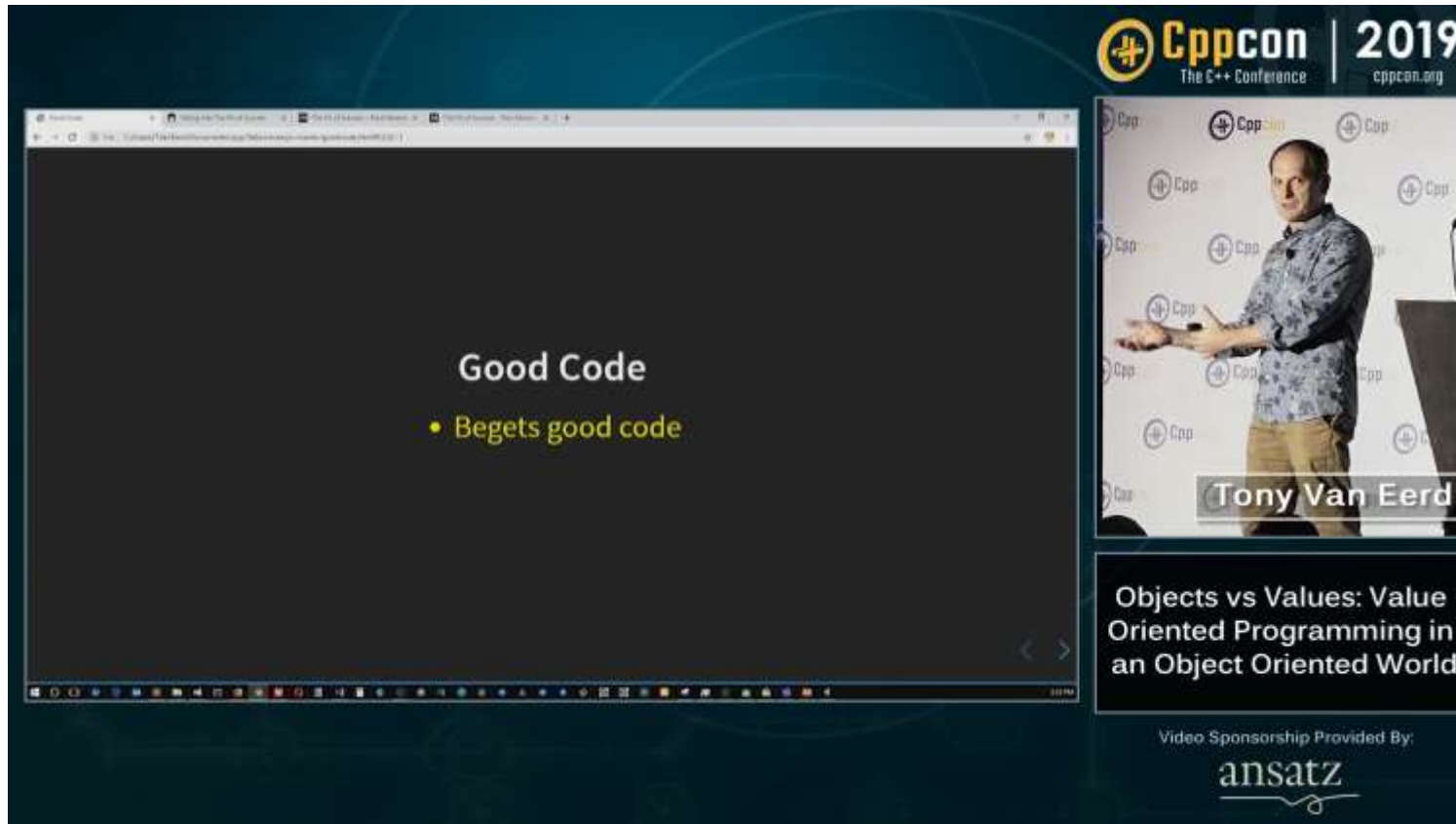
Unnecessary work is found. The fastest code is the code that doesn't exist



They ease code review

More issues caught at compile time = fewer issues to find at review time

“Good code begets good code”



... but...



Some aren't worth it

Number of emissions heavily outweigh their usefulness



More hits -> more useful

A warning's usefulness is usually proportional to how often it fires



More LOC -> More pain

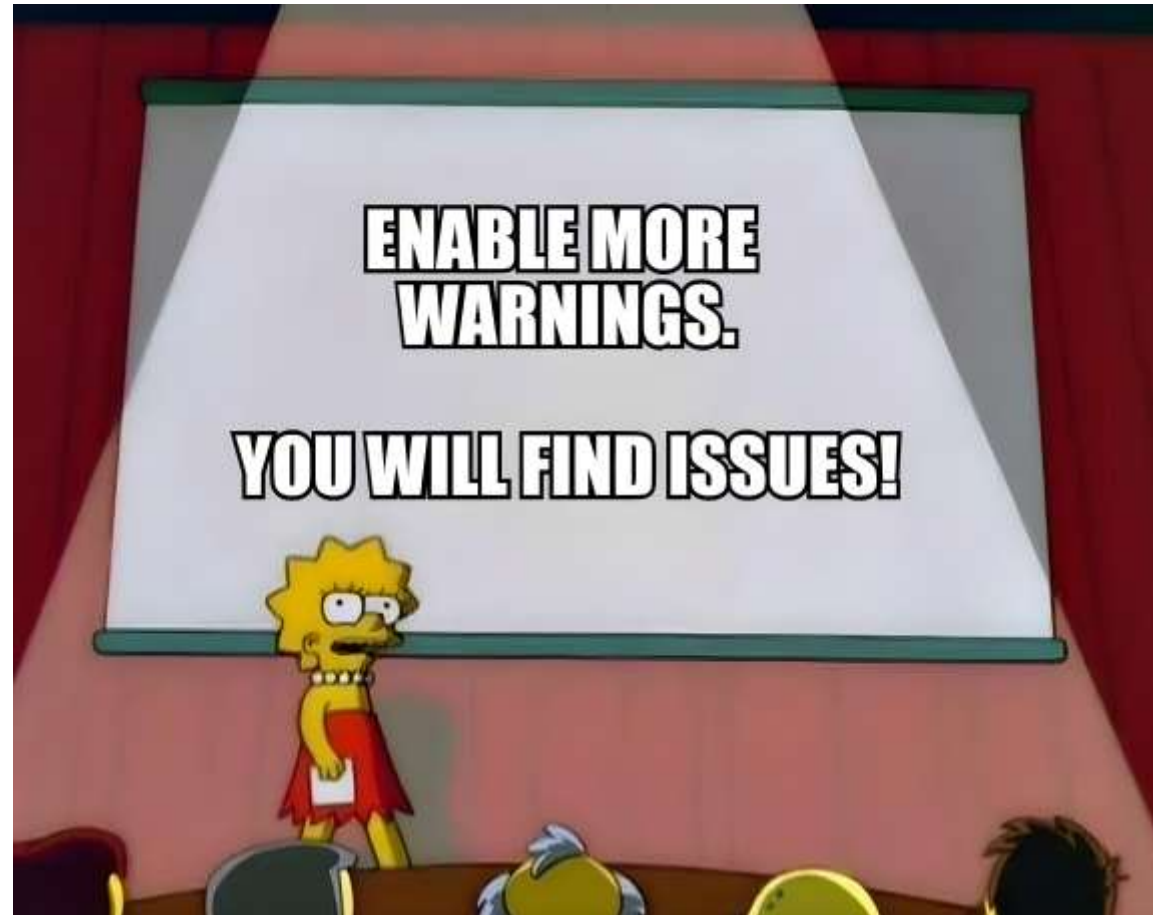
Trying to raise the warning level after the project is underway can be painful – Jason Turner



Moving target

New compiler and standard versions keep introducing new warnings

A call to action



Worth improving for UE 6?



Microsoft at CppCon 2026

Five sessions from Microsoft. Come find us.

MON Sep 14	15:15 - 16:15	The Journey to “/W4 /WX”: How Hard Could It Be? Keith Stockdale · GameDev	You are here
MON Sep 14	16:45 - 17:45	C++ in the Age of AI: How Visual Studio Is Evolving David Li, Augustin Popa · General	
TUE Sep 15	09:00 - 10:00	What’s New for C++ in Visual Studio Code: C++ and AI Tooling for Your Build, Your Code, Your Workflow Sinem Akinci, Ben McMorran · Tooling & Ecosystem	
WED Sep 16	14:00 - 15:00	Turbocharging Native Code Performance: Compiler Optimizations, Profile-guided Optimizations, and Beyond Eric Brumer · Tooling & Ecosystem	
THU Sep 17	15:15 - 16:15	The C++ Safety Issues Your Tools Can’t See Ion Todirel · Robotics & AI	

Q&A



Clever Programming Pun

About Blog Portfolio Talks

Search...

CTRL K



Keith Stockdale

C++, GPU Programming, Code Quality,
And maybe some other things...

Featured Blog Post

Validating Callable Parameters

#c++ #c++20 #c++23 #concepts #meta-programming #templates #reflection

I talk about implementing a way of inspecting the parameters of callables in C++20 and with C++

[Read more](#)

May 31, 2023

Latest Blog Post

ACCU On Sea 2026 Trip Report

#trip report #conference #c++

My trip report for ACCU On Sea 2026. I chat about the conference, the people I met, some of the I own talk.

[Read more](#)

July 18, 2023

